Democratic and People's Republic of Algeria

الجمهورية الجزائرية الديمقراطية الشعبية

Ministry of Higher Education and Scientific Research

وزارة التعليم العالي و البحث العلمي

جامعة العلوم والتكنولوجيا هواري بومدين

University of Science and Technology Houari Boumediene

Faculty of Informatics

Thesis for graduation

for the obtainment of the Bachelor degree

Option: ACAD

Design and Implementation of an Automatic and Intelligent IT Resource Provisioning Solution (Servers and Virtual Machines) for the Scalable Deployment of a High-Load Web Application.

Realized by:

Mr. Zakaria Hammal

Mr. Oussama Selatenia

Mr. Oussama Mennour

Mr. Younes Djeha

Supervised by:

Mr. Khaled Zeraoulia

Held on [DATE], before the jury composed of:

Mr. Lastname Firstname -- President of jury

Mr. Lastname Firstname -- Examiner

Promotion: [YEAR/YEAR]

Code PFE: [CODE]

Dedication

``

To our families, whose patience and encouragement sustained this work through its most demanding stages.

To our mothers, our fathers, our brothers and sisters — your sacrifices made ours possible.

To every colleague who shared ideas, debugged midnight sessions, and believed in this project.

Thank you.

''

The Authors

Acknowledgements

First and foremost, we render thanks to Allah, the Almighty, for granting us the strength, patience, and clarity of mind required to bring this work to completion.

We express our profound gratitude to our supervisor, **Mr. Khaled Zeraoulia**, for his invaluable guidance, availability, and constructive criticisms throughout the elaboration of this project. His technical acumen and pedagogical approach were decisive in shaping the rigour of this work.

We extend sincere thanks to the members of the jury for the honour they bestow upon us by dedicating time to evaluate and enrich this thesis.

Finally, we acknowledge the contributions, direct and indirect, of the teaching staff of the Faculty of Informatics at the University of Science and Technology Houari Boumediene (USTHB), whose courses furnished the theoretical foundations upon which this work is built.

Abstract

Modern healthcare information systems impose stringent requirements on their underlying infrastructure: continuous availability, elastic scalability under unpredictable load, and strict inter-tenant isolation. This thesis presents the design and implementation of an autonomous, intelligent resource provisioning framework for a cluster hosting a distributed medical information system.

The infrastructure is structured across four hierarchical layers: physical hardware, virtualisation, container orchestration, and an overarching observability stratum. At the virtualisation tier, a hybrid placement algorithm combining the Whale Optimisation Algorithm (WOA) and Differential Evolution (DE) governs the mapping of virtual machines to physical hosts, optimising a multi-objective fitness function that minimises CPU, RAM, I/O, and energy resource gaps in a single weighted objective. Container scheduling within virtual machine fleets is directed by a Best-Fit heuristic designed to maximise bin packing efficiency and reduce fragmentation. Automated scaling --- both vertical and horizontal --- is driven by telemetry-derived alerts with configurable severity thresholds.

An artificial intelligence layer augments rule-based orchestration with a gradient boosted predictive model (XGBoost) that dynamically recalibrates the placement algorithm's fitness weights according to the cluster's current structural bottleneck. A dual-agent cognitive system, composed of a Planner and an independent Critic, executes remediation actions under a risk-based escalation policy. Low- and medium-risk actions are executed autonomously; high-risk operations are deferred to an administrator via an asynchronous email approval workflow, with the entire decision chain recorded in an immutable audit log.

The system is evaluated in a proof-of-concept deployment of a microservices-based medical information system comprising ministry- and hospital-tier services, partitioned into five logically isolated Kubernetes namespaces and three network segments.

Keywords: cloud computing, virtualisation, container orchestration, microservices, Whale Optimisation Algorithm, Differential Evolution, Kubernetes, Prometheus, autonomous provisioning, AI-driven orchestration.

Résumé

Les systèmes d'information de santé modernes imposent des contraintes sévères à leur infrastructure : disponibilité continue, scalabilité élastique et isolation stricte entre locataires. Ce mémoire présente la conception et la réalisation d'un cadre autonome et intelligent de provisioning de ressources pour un cluster hébergeant un système d'information médical distribué.

L'infrastructure est structurée en quatre couches hiérarchiques. Un algorithme hybride WOA-DE gouverne le placement des machines virtuelles sur les hôtes physiques en minimisant une fonction de fitness multi-objectif. L'ordonnancement des conteneurs est assuré par une heuristique Best-Fit. Un modèle XGBoost recalibre dynamiquement les poids du placement selon le goulot d'étranglement courant du cluster. Un système dual-agent (Planificateur / Critique) exécute les actions de remédiation sous une politique d'escalade fondée sur le risque.

Mots-clés : virtualisation, orchestration de conteneurs, microservices, Whale Optimisation Algorithm, Differential Evolution, Kubernetes, Prometheus, provisioning autonome, orchestration par IA.

Contents

List of Figures

List of Tables

General Introduction

The progressive digitisation of critical public services --- healthcare in particular --- has imposed unprecedented demands on the software infrastructures that underpin them. A regional health information system must coordinate diagnostic records, pharmaceutical prescriptions, inter-hospital referrals, and administrative workflows across a geographically dispersed set of institutions, often under time-sensitive conditions where a processing delay translates directly into degraded patient care. These operational constraints place scalability, fault tolerance, and security at the centre of any credible technical design.

The emergence of virtualisation and container orchestration technologies has fundamentally altered the economics and operational logic of such infrastructures. Where previously a dedicated physical server was required for each application service, a modern cluster can consolidate hundreds of isolated workloads onto a shared hardware fleet, responding dynamically to load fluctuations without human intervention. Yet this operational flexibility is not freely obtained. Distributed systems introduce complexity that static provisioning models cannot address: an improperly placed workload propagates resource contention; a rigidly defined alert threshold fires at the wrong granularity; and a naive scaling trigger may inadvertently amplify the failure it was designed to contain.

This thesis investigates the architectural and algorithmic means by which this complexity can be brought under intelligent, automated control. The central question is the following: how can a cluster infrastructure spanning physical hardware, virtualisation, and container orchestration both maximise resource efficiency and guarantee continuous service availability for a high-load distributed application, while maintaining a security posture compatible with sensitive health data?

To address this question, three primary objectives were pursued. The first is to design a hierarchical, layer-separating cluster architecture that establishes explicit failure domains and network isolation boundaries without sacrificing operational flexibility. The second is to develop and integrate a multi-objective optimisation algorithm --- the hybrid WOA-DE --- that governs virtual machine placement with awareness of CPU, RAM, I/O, and energy constraints within a single fitness objective. The third is to build an autonomous orchestration layer, grounded in machine learning and dual-agent reasoning, capable of detecting emerging resource pathologies and executing calibrated remediation actions within a risk-bounded decision policy.

The document is organised as follows. Chapter 1 surveys the state of the art across the relevant domains: hardware virtualisation models, containerisation architectures,

container orchestration platforms, and prior work on virtual machine placement algorithms, situating the present contribution within the existing literature. Chapter 2 presents the general conceptual design of the proposed infrastructure in a technology-agnostic manner, articulating the four-layer architectural model, the WOA-DE placement algorithm and its fitness function, the monitoring and alert management subsystem, and the dual-agent AI framework with its associated security constraints. Chapter 3 grounds this design in a concrete implementation, documenting the specific technology choices made at each architectural layer and describing the deployed medical information system that serves as the proof-of-concept workload. Unavoidably, Chapter 3 remains partially incomplete at the time of writing, pending finalisation of the deployment environment; all such gaps are explicitly labelled.

Chapter 1

State of the Art

1.1 Introduction

Virtualised, containerised infrastructure management has generated a substantial research literature over the past two decades, spanning hypervisor engineering, container runtime design, distributed orchestration, and combinatorial resource allocation. Surveying this body of work is not merely a formality: the specific design decisions in Chapter 2 --- the choice of a hybrid placement strategy, the three-segment network model, the two-tier agent architecture --- each respond to identifiable shortcomings in prior approaches. This chapter maps those shortcomings through structured comparative analyses of the relevant technology families and closes with a critical synthesis that locates the precise gap this thesis addresses.

1.2 Hardware Virtualisation

1.2.1 Hypervisor Classification

Two primary hypervisor architectures govern the landscape of hardware virtualisation. Type 1 hypervisors --- often termed bare-metal hypervisors --- execute directly on the physical hardware without an intervening host operating system. This direct hardware access yields lower scheduling latency and higher throughput. Type 2 hypervisors run as processes within a conventional host operating system, trading raw performance for installation simplicity and portability. Table 1.1 contrasts the defining characteristics of each class.

1.2.2 Virtual Machines versus Containers

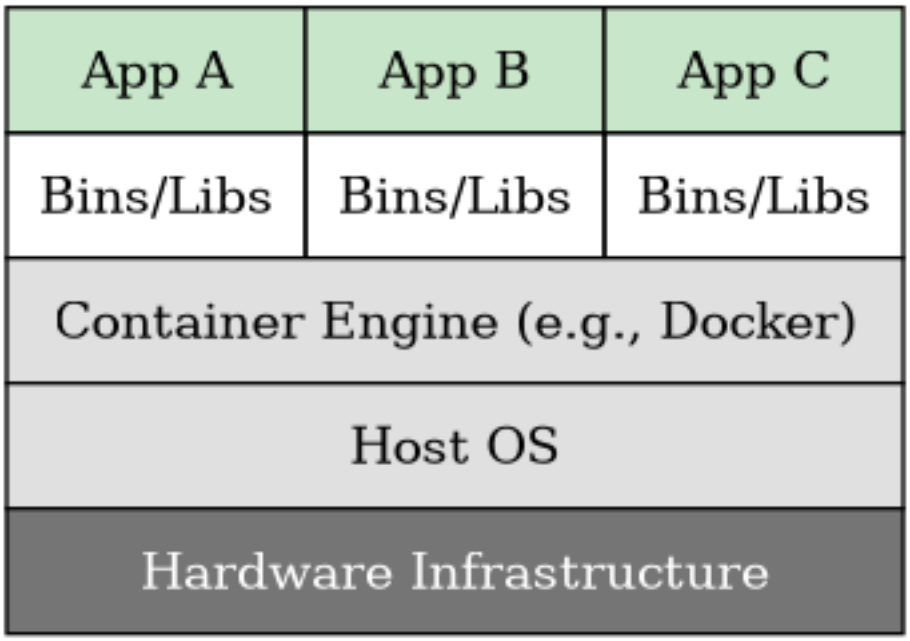
Virtual machines and containers both deliver workload isolation, yet they operate at fundamentally different levels of the system stack. A virtual machine encapsulates an entire operating system, whereas a container shares the host kernel and isolates only the user-space environment. Table 1.2 presents the principal trade-offs between these two paradigms.

Table 1.1: Comparative analysis of Type 1 and Type 2 hypervisors.

Criterion	**Type 1 (Bare-Metal)**	**Type 2 (Hosted)**
Execution layer	Directly on hardware; no host OS	As a process within a host OS
Performance	Near-native; minimal overhead	Additional OS scheduling layer
Security isolation	Strong; no shared host OS kernel	Weaker; host compromise affects all guests
Typical use case	Production data centres, enterprise deployments	Developer workstations, testing environments
Examples	VMware ESXi, Microsoft Hyper-V, Xen, KVM, Proxmox VE	VirtualBox, VMware Workstation, QEMU (user mode)
Hardware access	Direct, via device drivers in hypervisor	Via host OS device driver stack
Deployment cost	High; requires dedicated hardware	Low; runs on existing hardware

Table 1.2: Virtual machines versus containers: a comparative analysis.

Criterion	**Virtual Machines**	**Containers**
Isolation unit	Full guest OS + application	Process namespaces + cgroups
Startup time	Tens of seconds to minutes	Sub-second to a few seconds
Image size	Gigabytes (full OS)	Megabytes to a few hundred MB
Kernel sharing	Independent kernel per VM	Shared host kernel
Security boundary	Strong hardware-enforced boundary	Weaker; kernel vulnerabilities affect all
Resource overhead	High (hypervisor + guest OS)	Low (thin runtime layer)
Density per host	Tens of VMs per host	Hundreds of containers per host
Portability	Image-level (OVA, VMDK)	Image-level (OCI/Docker)
Persistent storage	Virtual disk files	Volumes external to container



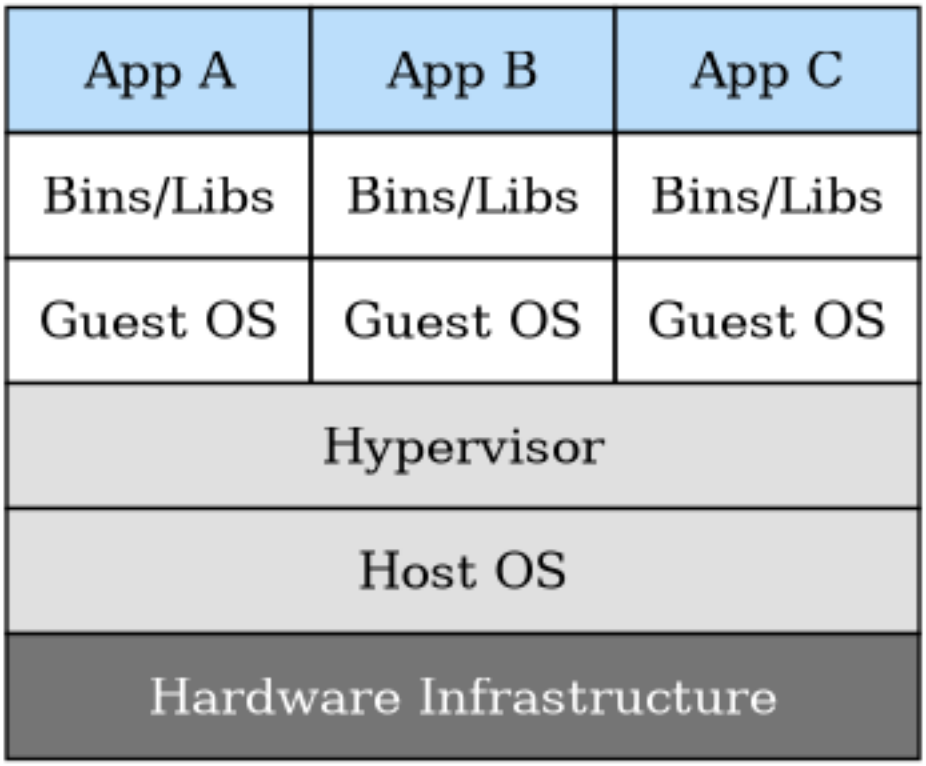


Figure 1.1: Architectural comparison between hardware virtualisation (Virtual Machines) and operating-system-level virtualisation (Containers). Source: adapted from open documentation.

1.3 Software Architecture: Monolithic vs Microservices

The structural design of the application layer profoundly conditions the scalability of the underlying infrastructure. A monolithic application consolidates all business logic, data access layers, and interface components into a single deployable artefact. A microservices architecture, by contrast, decomposes the application into a collection of autonomous services, each encapsulating a single bounded business capability and communicating via well-defined interfaces. Table 1.3 summarises the operational trade-offs between these two paradigms.

Table 1.3: Monolithic architecture versus microservices: comparative analysis.

Criterion	**Monolithic**	**Microservices**
Scalability	Entire application replicated; coarse-grained	Per-service scaling; fine-grained
Deployment	Single artefact; atomic but risky	Independent per-service deployments
Fault isolation	Single point of failure for all functions	Graceful degradation; faults contained
Technology stack	Uniform across entire codebase	Polyglot; per-service choice
Inter-module coupling	Tight; function calls within process	Loose; network API contracts
Operational complexity	Low; single process to monitor	High; distributed tracing required
Release velocity	Slows as codebase grows	High; independent release cycles
Data management	Single shared database	Database-per-service pattern
Team structure	Centralised; all teams share codebase	Autonomous teams per service

1.4 Container Orchestration Platforms

As the number of containerised services grows beyond the capacity of manual administration, orchestration platforms become essential. Table 1.4 compares the three principal open-source container orchestration systems.

Table 1.4: Comparison of leading container orchestration platforms.

Criterion	**Kubernetes**	**Docker Swarm**	**HashiCorp Nomad**
Architecture	Control plane + worker nodes; etcd state store	Manager + worker nodes; Raft consensus	Client-server; Raft consensus
Workload types	Containers, Jobs, StatefulSets, DaemonSets	Services, Tasks	Containers, JVM apps, scripts
Auto-scaling	HPA, VPA, KEDA	Manual or external	Autoscaler via external plugin
Networking	CNI plugins (Calico, Flannel, Cilium)	Overlay (VXLAN-based)	CNI plugins
Storage	PersistentVolume / StorageClass abstractions	Volume mounts	Host volumes, CSI plugins
Learning curve	High	Low to medium	Medium
Community maturity	Very high; CNCF flagship	Declining; Docker-native	Growing
Multi-tenancy	Namespaces + RBAC + NetworkPolicy	Stack isolation only	Namespaces + ACL

1.5 Virtualisation Platforms

The choice of virtualisation platform determines the capabilities available for VM lifecycle management, high availability, and integration with the orchestration layer above it. Table 1.5 compares the leading open-source and commercial alternatives.

Table 1.5: Comparison of virtualisation platforms.

Criterion	**Proxmox VE**	**VMware vSphere**	**OpenStack**
Licence	Open-source (AGPL) with enterprise subscription option	Proprietary; per-socket licensing	Open-source (Apache 2.0)
Hypervisor	KVM (QEMU) + LXC	ESXi (proprietary)	KVM, Xen, HyperV (pluggable)
Management UI	Web-based; REST API	vCenter; REST API	Horizon dashboard; REST APIs
Cluster features	HA, live migration, shared storage, clustering	DRS, HA, vMotion, vSAN	Nova scheduling, live migration
Storage integration	ZFS, Ceph, iSCSI, NFS, local LVM	vSAN, NFS, iSCSI, vVols	Cinder (block), Swift (object)
Container support	LXC natively; Docker via VMs	VMware Tanzu (additional cost)	Via Nova or Kubernetes integration
On-premises suitability	Excellent; low cost	Excellent; high cost	Suitable for large deployments
Learning curve	Moderate	High	Very high

1.6 VM Placement Algorithms in the Literature

Virtual machine placement constitutes a combinatorially complex optimisation problem whose difficulty scales with the number of physical hosts, candidate VMs, and optimisation objectives. Several algorithmic families have been explored in the literature. Table 1.6 presents a non-exhaustive review.

Table 1.6: VM placement algorithms from the literature.

Algorithm family	**Examples**	**Optimisation objective**	**Key characteristics**
Bin-packing heuristics	First-Fit, Best-Fit, Worst-Fit	Minimise active hosts	Fast; no convergence guarantee; greedy
Evolutionary / genetic	GA, DE [3]	Multi-objective resource utilisation	Population-based; good exploration; computationally heavier
Swarm intelligence	PSO, WOA [2], Bat Algorithm	Energy + load balance	Bio-inspired; prone to premature convergence in isolation
Reinforcement learning	DQN, PPO, CILP [12]	Long-horizon energy savings	Learns online; high training cost; hard to interpret
Multi-criteria (AHP)	AHP-DRS [11]	Weighted multi-attribute scoring	Transparent weighting; expert knowledge required
Hybrid (WOA + DE)	This work	CPU + RAM + I/O + energy	Combines exploitation depth (WOA) with diversity maintenance (DE)

1.7 Critical Synthesis: Open Problems in VM Placement

The algorithm families surveyed in Table 1.6 share a recurring set of unresolved limitations that motivate the design decisions in Chapter 2.

Single-objective optimisation. Bin-packing heuristics (First-Fit, Best-Fit) minimise the count of active hosts but are blind to energy consumption and network traffic patterns. Saxena et al. [7] introduce security as a second objective; Yang et al. [14] add task structure; none integrates CPU, RAM, I/O, and energy simultaneously under a normalised, dynamically weighted objective. The fitness function proposed in Section 2.5 addresses this directly.

Premature convergence in isolation. Pure WOA [2] concentrates the population around the current best solution within a few dozen iterations on high-dimensional placement problems, where the search space grows exponentially with the number of hosts. The convergence behaviour is well-documented and is the primary motivation for hybridisation. DE [3] avoids this collapse through algebraic mutation but converges slowly on exploitation-heavy problems. Neither algorithm alone is adequate for the time-sensitive provisioning loop of an on-line cluster.

Static thresholds. Even systems with sound placement algorithms --- including Lai et al. [4] and Alahmad and Agarwal [15] --- rely on fixed resource thresholds to trigger migrations. Fixed thresholds assume stationary workload distributions, an assumption that fails for microservices deployments subject to diurnal traffic cycles and sudden load spikes. Saxena and Singh [16] propose a neural network for dynamic threshold prediction but provide no mechanism to recalibrate the placement fitness function accordingly.

Absence of safety-bounded autonomous execution. The works reviewed address placement and scaling as algorithmic problems, but none proposes a governance framework for autonomous remediation. Tuli et al. [12] demonstrate online learning for provisioning but treat execution as unrestricted; the consequences of an incorrectly predicted action on a production cluster are not considered. The dual-agent system with monotonic risk escalation developed in Section 2.8 is a direct response to this gap.

Infrastructure-as-monolith assumption. Most placement papers treat the cluster as a flat pool of physical machines. The mini-cluster partitioning proposed in Section 2.5 departs from this model: by partitioning the search space along application dependency boundaries, scheduling complexity is reduced from $O(n)$ to $O(n/k)$ and the majority of inter-service traffic is confined within a single logical segment.

1.8 Related Works

Table 1.7 synthesises the contributions most directly pertinent to the present thesis. Each entry corresponds to a bibliographic reference cited in the following chapters.

Table 1.7: Related works: synthesis table.

Authors	Year	Contribution	Methodology	Strengths	Limitations / Relevance
Mirjalili & Lewis [2]	2016	Introduces the Whale Optimisation Algorithm (WOA)	Bio-inspired; bubble-net hunting model	Strong exploitation; simple parameter set	Premature convergence; foundational reference for WOA component
Storn & Price [3]	1997	Differential Evolution for global optimisation	Population-based algebraic mutation and crossover	Excellent exploration; monotonic improvement	Requires continuous search space; provides DE component
Zhang, Zhou & Wang [5]	2020	Intelligent optimal VM allocation in cloud data centres	Multi-objective evolutionary optimisation	Demonstrated applicability to VM placement	Cloud-centric; limited energy awareness
Saxena et al. [7]	2022	Secure and multi-objective VM placement framework	Multi-objective optimisation with security constraints	Combines performance and security objectives	Does not address AI-driven dynamic weight recalibration
Saxena & Singh [16]	2022	Proactive autoscaling with neural network VM allocation	Online multi-resource neural network	Proactive approach; energy-efficient	Neural network interpretability limitations

Continued on next page

Table 1.7 continued...

Authors	Year	Contribution	Methodology	Strengths	Limitations / Relevance
Alahmad & Agarwal [15]	2024	Dynamic VM placement for application availability	Multi-objective dynamic placement in cloud networks	Availability-aware; dynamic adaptation	Does not handle on-premises cluster topology
Tuli, Casale & Jennings [12]	2023	Co-simulation imitation learning for resource provisioning	Reinforcement learning via co-simulation	Handles dynamic environments; learns online	High training overhead; black-box decisions
Senjab et al. [9]	2023	Survey of Kubernetes scheduling algorithms	Systematic literature review	Comprehensive overview of K8s scheduling	Survey only; no novel algorithm
Lai, Zheng & Lu [4]	2025	Energy-aware VM migration with dual resource thresholds	Threshold-based live migration policy	Energy savings with performance guarantees	Dual-threshold approach; informs scaling thresholds
Begam, Wang & Zhu [6]	2020	Time-sensitive VM provisioning in resource-constrained clouds	Bin-packing with time constraints	Addresses time-sensitive workloads	Limited multi-objective scope
Yang et al. [14]	2025	Task-driven VM optimisation placement model	Task-aware placement optimisation	Task-specific resource allocation	Does not address AI-based dynamic adaptation

Continued on next page

Table 1.7 continued...

Authors	Year	Contribution	Methodology	Strengths	Limitations / Relevance
Gu et al. [11]	2023	Multi-decision AHP VM scheduling in cloud platforms	AHP-based multi-criteria scheduling	Transparent decision process; multi-criteria aware	Expert-dependent weight assignment
Nguyen, Phan & Kim [13]	2022	Load balancing of Kubernetes-based edge infrastructure	Resource-adaptive proxy for K8s edge clusters	Practical K8s load distribution	Edge-specific; limited to proxy layer
Bompotas, Kalogeropoulos & Makris [10]	2025	Multi-purpose commodity hardware cluster design	Cluster architecture with commercial off-the-shelf hardware	Low-cost cluster feasibility	General purpose; no placement optimisation
Balabanov & Apostolov [1]	2025	Data centre infrastructure monitoring standards	DCIM methodology survey	Comprehensive monitoring taxonomy	Monitoring scope only; no placement
Sung, Han & Kim [8]	2024	VM pre-provisioning via cloning for edge cloud	Clone-based horizontal scaling	Fast horizontal scaling via snapshot cloning	Edge context; informs clone-based scaling strategy

1.9 Conclusion

Four structural gaps emerge from this review. First, no existing placement work combines all four resource dimensions --- CPU, RAM, I/O, and energy --- under a single normalised objective whose weights are recalibrated at runtime by a predictive model; approaches addressing multiple objectives fix their weight vectors at design time. Second, hybrid evolutionary strategies that combine WOA's exploitation geometry with DE's population diversity consistently avoid the convergence failure of single-paradigm

bio-inspired optimisers, yet no published placement system applies this combination to an on-premises, on-line provisioning scenario. Third, the migration from static to dynamic alert thresholds has been studied at the monitoring layer but not connected back to the placement layer: no system feeds predicted bottleneck classifications into a placement fitness recalibration loop. Fourth, autonomous remediation in production clusters has been demonstrated only without governance constraints; the question of how to bound an agent's authority to prevent irreversible damage has not been formally addressed.

The architecture proposed in Chapter 2 targets precisely these four gaps. The WOA-DE hybrid eliminates premature convergence at the placement layer. The multi-objective weighted Euclidean fitness with Z-score normalisation integrates all four resource dimensions. The XGBoost prediction layer feeds dynamically adjusted weights back to the placement algorithm. The Planner/Critic dual-agent system with monotonic risk escalation provides autonomous execution with bounded authority. The experimental system described in Chapter 3 constitutes the first integrated realisation of all four mechanisms on a single on-premises cluster.

Chapter 2

Conception

2.1 Introduction

A cluster capable of sustaining a high-load distributed application under the competing imperatives of resource efficiency and continuous availability cannot be designed at a single abstraction level. Decisions at the hardware layer constrain what the virtualisation layer can offer; those decisions constrain what the container scheduler can do; and the scheduler's behaviour determines what the monitoring layer must observe and correct. Each layer's design is therefore inseparable from those above and below it.

This chapter develops the conceptual design of the proposed infrastructure from the ground up, working through the physical and network foundations, the VM placement algorithm and its mathematical basis, the container scheduling heuristic, the telemetry and alert management subsystem, and the autonomous decision-making architecture with its security model. No specific product, programming language, or tool is named; Chapter 3 maps each element to its concrete realisation.

2.2 Design Objectives and Validation Criteria

2.2.1 Competing Requirements

Physical hardware is capital-intensive. A cluster that reserves capacity against every conceivable failure mode is a cluster that wastes most of its hardware most of the time. Conversely, a cluster that maximises utilisation by packing workloads as densely as possible amplifies the impact of any single node failure: more services share the same failure domain, and the margin available for migrating disrupted workloads shrinks toward zero. This opposition between *resource efficiency* and *continuous availability* is the central design constraint of the proposed infrastructure [14, 15].

The architecture does not eliminate this tension; it manages it. Dense placement is maintained at the default operating point, but the scheduler maintains awareness of failure-domain boundaries, so replicas of the same service are never co-located on hosts that share a common failure mode. Vertical and horizontal scaling mechanisms can expand resource allocation in response to pressure without requiring over-provisioned

headroom at rest. The hosted application is required to follow a decomposed service model so that load on one component can be scaled without touching the rest [9].

2.2.2 Proof-of-Concept Validation Conditions

Four measurable conditions jointly define a successful proof-of-concept. Each condition targets one behavioural property that static provisioning models cannot guarantee.

Automated provisioning: a new workload host must be instantiated and onboarded without direct operator intervention. Satisfying this condition confirms that the placement algorithm and its supporting API layer function as a closed loop rather than as a decision-support tool.

Reactive remediation: the monitoring subsystem must detect an injected load anomaly --- a synthetic CPU saturation event or an elevated HTTP error rate --- and dispatch the correct remediation within a bounded time window. The time budget is defined by the scrape interval of the telemetry system plus the agent reasoning cycle; total latency from threshold breach to executed action must remain below two minutes.

Scheduling correctness: the container scheduler must respect all declared resource limits during distribution and rebalancing. Any placement that would cause a virtual machine to exceed its allocated RAM or CPU envelope is classified as a scheduler failure, regardless of whether the service remains available.

Continuous reachability: the hosted application must remain accessible to external HTTP clients throughout all automated provisioning and scaling operations. Any downtime interval exceeding the duration of a single health-check cycle is classified as a high-availability failure.

2.3 Layered Infrastructure Architecture

Four architectural layers are superimposed in the proposed cluster. The boundaries between layers are hard: each layer exposes a defined interface to the layer above and has no knowledge of the layers above it. This separation prevents a fault or design change at one layer from cascading unpredictably through the rest of the stack.

The **physical layer** supplies raw computing capacity --- processors, memory, persistent storage, and interconnect --- and is the only layer with direct contact with hardware. All decisions about failure-domain isolation and network topology are made here.

The **virtualisation layer** partitions physical capacity into isolated virtual machines with defined resource envelopes, abstracting away hardware heterogeneity. The placement algorithm operates at this layer, matching resource requests to available physical capacity.

The **containerisation layer** distributes application workloads as containers across the VM fleet. Scheduling, service discovery, and horizontal scaling are managed at this layer; the underlying VM topology is opaque to it.

The **monitoring layer** is the only layer that observes all three beneath it. It does not run application workloads. Its function is to ingest telemetry, evaluate it against defined thresholds, and emit remediation commands downward when conditions warrant. It is represented in Figure 2.1.

2.4 Physical and Network Foundation

2.4.1 Physical Infrastructure and Failure Domains

The physical layer comprises a fleet of servers interconnected by a managed switching fabric that enforces bounded latency for intra-cluster communication. At the cluster perimeter, stateful packet inspection filters unauthorised ingress traffic before it reaches any internal resource. Core routing equipment manages inter-segment traffic and public internet connectivity.

To satisfy availability constraints, the physical layer is designed around explicit *failure domains*. A failure domain is defined as a group of servers sharing a common point of vulnerability --- a shared power distribution unit, a single top-of-rack switch, or a single physical host. Higher-level scheduling algorithms are programmed to distribute workload replicas across distinct failure domains; a hardware event affecting one domain therefore cannot eliminate all instances of a given service. All core routing and switching equipment is deployed in high-availability pairs to eliminate network-partitioning risk, as illustrated in Figure 2.2.

2.4.2 Overlay Network Architecture

Physical compute abstraction is insufficient for multi-tenant security without an equivalent abstraction at the network layer. An overlay network decouples the logical topology from the physical cabling, permitting dynamic segmentation without hardware reconfiguration.

Three-segment isolation model. The overlay network is partitioned into exactly three logical segments, corresponding to the three distinct trust domains present in the target deployment. The first segment hosts the workloads of tenant group A. The second hosts tenant group B. The third is reserved for the observability and monitoring infrastructure.

Three segments, rather than two or four, follow directly from the threat model. Merging the two tenant groups into a single segment would allow a compromised ser-

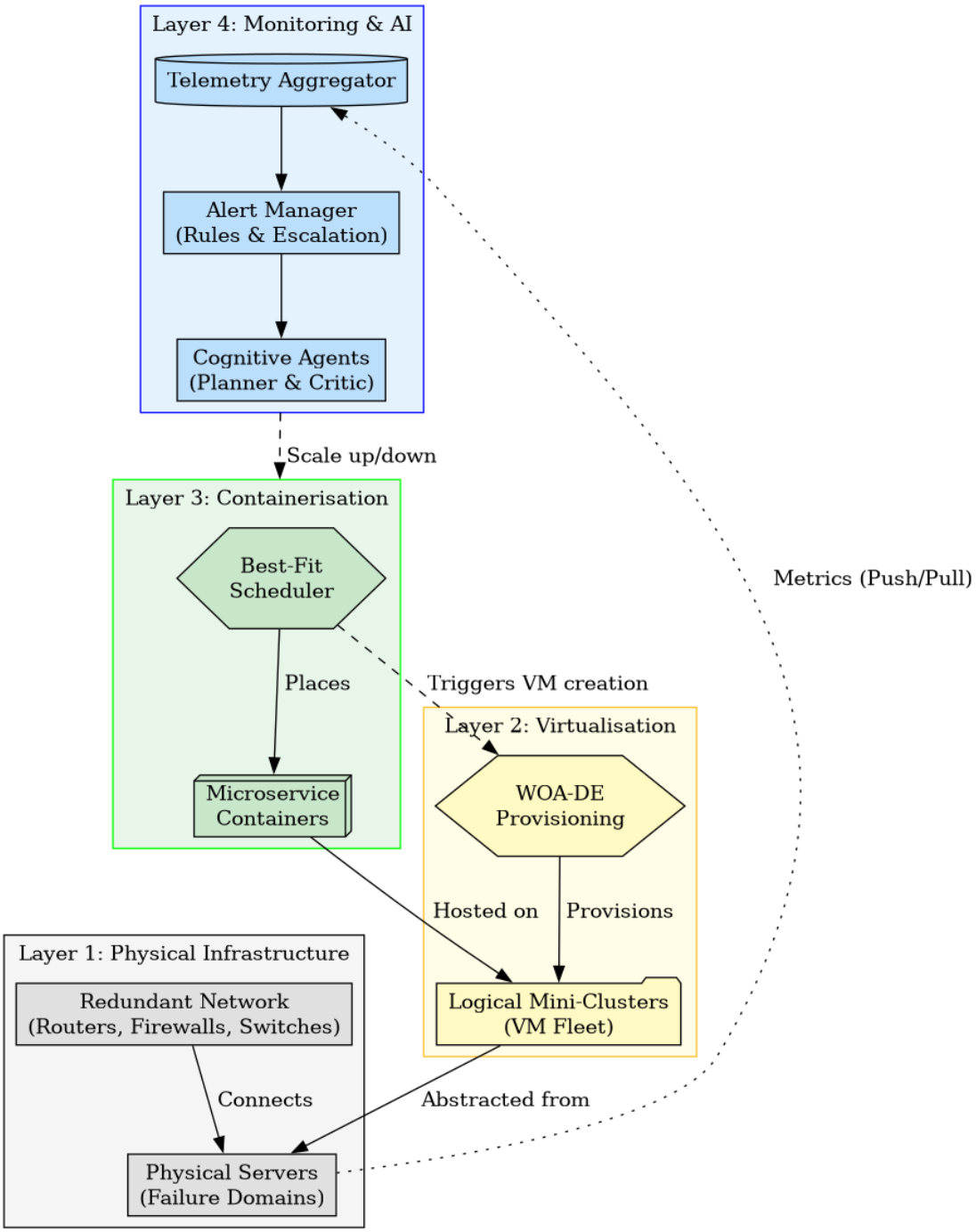


Figure 2.1: The hierarchical four-layer architecture of the cluster ecosystem, showing the data flow from physical infrastructure to the monitoring intelligence stratum.

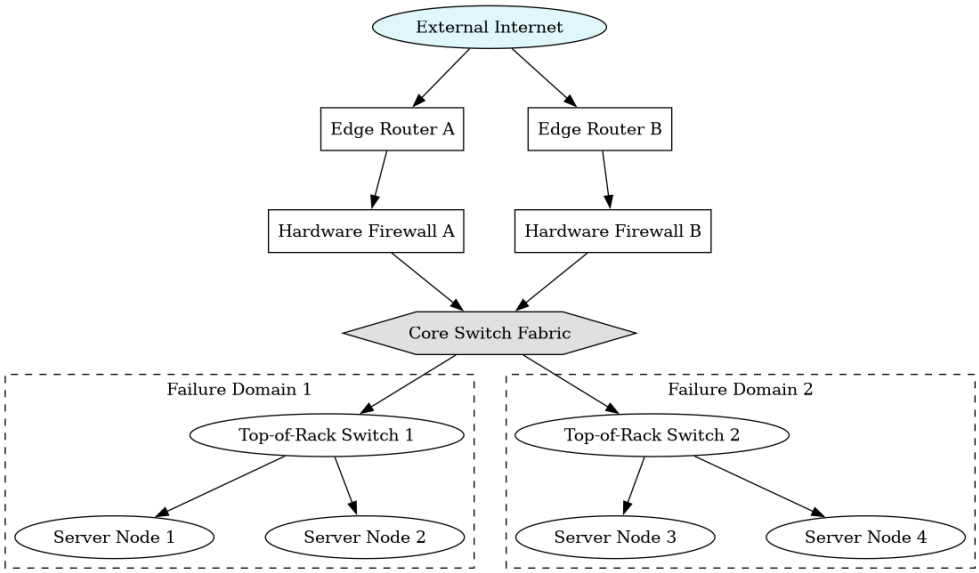


Figure 2.2: Physical infrastructure layer: explicit failure domains and redundant switching fabric design.

vice in one group to reach services in the other without crossing a firewall boundary --- unacceptable when the two groups operate under different administrative and organisational domains. Merging the monitoring segment into a tenant segment would expose telemetry data to tenant traffic and risk losing observability during a tenant-side incident. Subdividing beyond three would not add a distinct trust boundary; it would only add routing complexity. Three segments is therefore the minimal partitioning that satisfies the isolation requirement [7].

Routing and access control. All inter-segment traffic passes through a central, software-defined virtual router operating under a *default-deny* posture: any packet not matched by an explicit permit rule is silently discarded. Cross-segment communication requires bidirectional policy rules. The monitoring segment may initiate data collection connections into the tenant segments; no connection originating from a tenant segment toward the monitoring infrastructure is permitted. Outbound internet access is provided via network address translation, masking the internal cluster topology from external networks.

2.5 Virtualisation and VM Placement

2.5.1 Mini-Cluster Partitioning

The VM fleet is partitioned into k logical *mini-clusters*, where each mini-cluster groups VMs that host containers with strong mutual communication. The rationale for this decomposition is threefold.

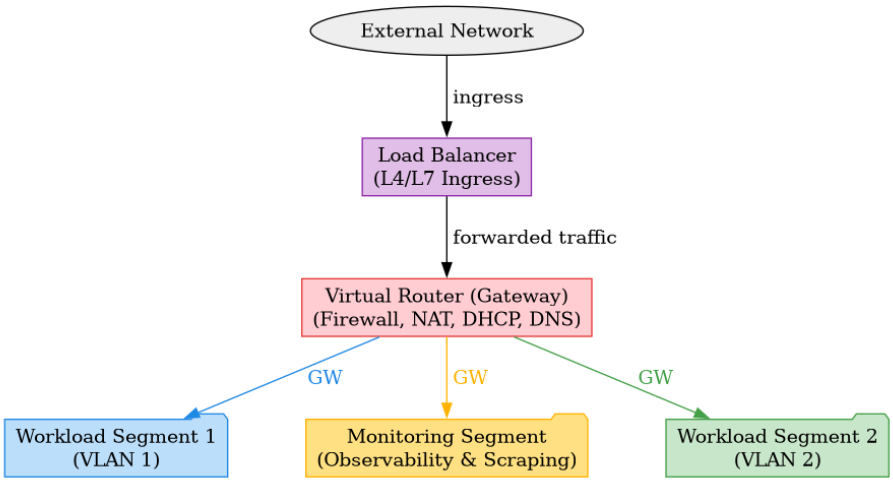


Figure 2.3: Overlay network topology: a central virtual router enforces default-deny policies and isolates the three logical segments by distinct trust level.

Scheduling complexity. Without partitioning, the placement algorithm evaluates n physical machines for every VM request. With k mini-clusters of roughly equal size, the search space per invocation shrinks to approximately $\lfloor n/k \rfloor$ machines, reducing scheduling latency proportionally. For on-line provisioning --- where placement decisions are made under live traffic --- this reduction is operationally significant [15].

Equitable resource distribution. Partitioning prevents a subset of high-demand VMs from monopolising the physical hosts nearest to the orchestration plane. Each mini-cluster attracts a bounded subset of the workload, distributing resource pressure across the full physical fleet.

Behavioural homogeneity. VMs within the same mini-cluster host services with similar access patterns and scaling behaviour. This homogeneity means that a scaling decision --- adding RAM, for instance --- generalises across the mini-cluster: the same physical target serves any VM in the group. This property becomes increasingly valuable as the number of VMs grows and individual case-by-case analysis becomes infeasible.

A cross-boundary VM --- one whose container must communicate heavily with containers in a different mini-cluster --- represents a partitioning error. In the static assignment strategy, this is prevented at design time by mapping the mini-cluster boundaries to the application's dependency graph; services with strong mutual affinity are grouped into the same mini-cluster before any VM is provisioned. A mispartitioned VM increases inter-segment network traffic, degrades placement algorithm efficiency (the most suitable host may lie outside the search space), and undermines the homogeneity property. Detection of systematic cross-boundary traffic is one of the triggers for dynamic repartitioning in the long-term evolution of the system.

Determining k. Two strategies govern the choice of k [12, 16]. In the **static** approach, k is fixed from the application's architectural dependency graph at design time; services sharing data paths are co-located in the same cluster. In the **dynamic** approach, a machine learning model continuously analyses inter-VM communication matrices and resource correlation patterns, adjusting cluster boundaries as the application evolves. The present implementation uses the static strategy; the dynamic approach is identified as a direction for future development.

2.5.2 The Hybrid WOA-DE Placement Algorithm

The VM provisioning algorithm addresses the following optimisation problem: given a request for a new virtual machine with a specified resource requirement vector, which physical machine within the target mini-cluster should host it? A hybrid algorithm integrating the **Whale Optimisation Algorithm** (WOA) [2] and **Differential Evolution** (DE) [3] is adopted. The application of evolutionary strategies to VM allocation is well established in the literature [5], and WOA has demonstrated particular efficacy in multi-objective placement settings [7].

Design rationale. WOA mathematically models the bubble-net hunting behaviour of humpback whales. Candidate solutions are updated by either contracting towards the best-known mapping (exploitation) or spiralling around it (local intensification). DE generates trial solutions via algebraic mutation and crossover of population members, providing superior spatial exploration. Pure WOA is vulnerable to premature convergence on local optima. The hybrid design mitigates this by splitting the candidate population at each iteration: one subset undergoes WOA position updates (deep exploitation) while the other undergoes DE mutation (diversity maintenance). This combination prevents stagnation while guaranteeing monotonic population improvement.

Input and output. The algorithm receives a resource request vector comprising the required virtual CPUs, memory, storage, and the target mini-cluster identifier, as well as a snapshot of current host metrics drawn from the monitoring layer. It outputs the recommended VM-to-host mapping and its associated fitness score.

Formal description. Let n denote the population size and T the maximum number of iterations. Each candidate solution X_i represents a complete mapping of virtual machines to physical hosts within the selected mini-cluster. The quality of a mapping is evaluated by the fitness function defined in Section 2.5.3.

Algorithm 1 presents the complete procedure.

Algorithm 1: Hybrid WOA-DE VM Placement Algorithm

Input : Resource request vector $(v_{\text{cpu}}, v_{\text{ram}}, v_{\text{io}})$; target mini-cluster $\mathcal{C}$; host metric snapshot $\mathcal{M}$; population size n; max iterations T; WOA spiral constant b; DE scale factor F; DE crossover rate $CR \in (0, 1)$

Output: Optimal VM-to-host mapping X^*; fitness score $f(X^*)$

`// Phase 1 - Initialisation`

Randomly generate n candidate mappings $\{X_0, X_1, \ldots, X_{n-1}\}$ within feasible placement bounds derived from $\mathcal{C}$ and $\mathcal{M}$

Evaluate $f(X_i)$ for all i

$X^* \leftarrow \arg\min_i f(X_i)$

Set $a_0 \leftarrow 2$

`// Phase 2 - Main evolutionary loop`

for $t \leftarrow 0$ **to** T **do**

  Compute $a \leftarrow 2(1 - t/T)$

  Draw $r_1, r_2 \sim \text{Uniform}(0, 1)$

  Compute $A \leftarrow 2a\,r_1 - a$ **and** $C \leftarrow 2\,r_2$

  `// WOA update sub-phase (first half of population)`

  foreach X_i *in WOA subset* **do**

    Draw $p \sim \text{Uniform}(0, 1)$

    if $p < 0.5$ **then**

      if $|A| < 1$ (exploitation) **then**

        $D \leftarrow |C \cdot X^* - X_i(t)|$

        $X_i(t{+}1) \leftarrow X^* - A \cdot D$

      else

        Select X_{rand} uniformly from population

        $D \leftarrow |C \cdot X_{\text{rand}} - X_i(t)|$

        $X_i(t{+}1) \leftarrow X_{\text{rand}} - A \cdot D$

    else

      `// Spiral bubble-net update`

      $D \leftarrow |X^* - X_i(t)|$

      Draw $l \sim \text{Uniform}(-1, 1)$

      $X_i(t{+}1) \leftarrow D\,e^{bl}\cos(2\pi l) + X^*(t)$

    Update X^* if $f(X_i(t{+}1)) < f(X^*)$

  `// DE mutation-crossover-selection sub-phase (second half)`

  foreach X_j *in DE subset* **do**

    Select distinct X_{r_1}, X_{r_2} from population

    $V_j \leftarrow X^* + F \cdot (X_{r_1} - X_{r_2})$ `// mutation`

    foreach *component* k **do**

      if $\text{rand}() \le CR$ **then**

        $U_j[k] \leftarrow V_j[k]$

      else

        $U_j[k] \leftarrow X_j[k]$

    if $f(U_j) < f(X_j)$ **then**

      $X_j \leftarrow U_j$

    `// greedy selection`

    Update X^* if $f(X_j) < f(X^*)$

return X^*, $f(X^*)$

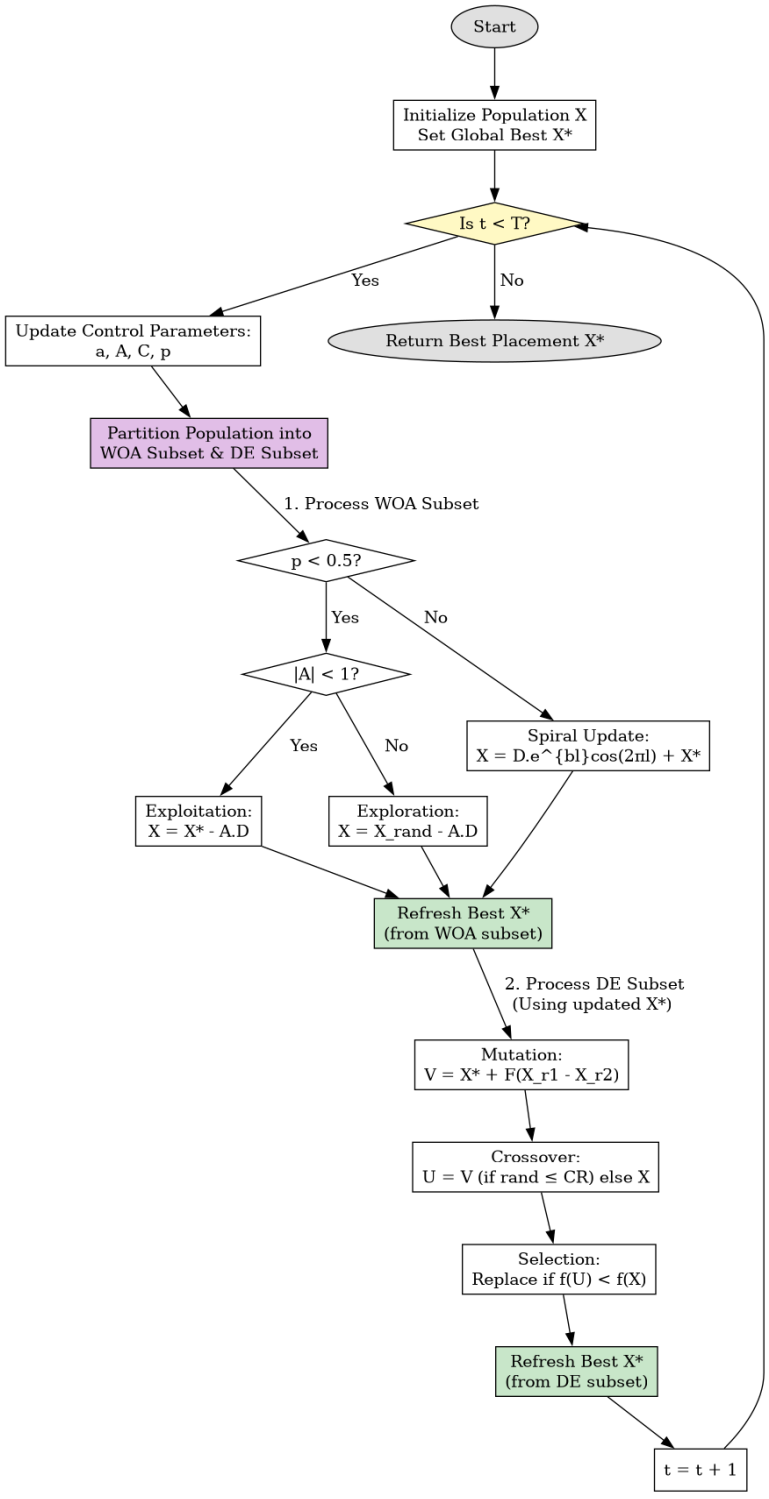


Figure 2.4: Flowchart of the hybrid WOA-DE provisioning algorithm, highlighting the parallel WOA and DE evaluation phases and the convergence condition.

Algorithmic properties. The dual exploration mechanics --- WOA's distance-based spiral geometry and DE's algebraic mutation --- jointly suppress local optima stagnation. The DE greedy selection phase guarantees monotonic population improvement, ensuring algorithmic convergence. Because both algorithms evaluate fitness analytically, they accommodate the discretisation required to map continuous numerical outputs back to feasible, discrete host assignments.

2.5.3 Fitness Function

Each candidate placement is evaluated by a fitness function that acts on four resource dimensions: CPU capacity (C), RAM (R), I/O throughput (IO), and energy consumption (E). The function must satisfy two design requirements that pull in opposite directions: it must treat all four dimensions equitably regardless of their physical units, and it must penalise near-exhaustion more aggressively than slight under-utilisation.

Normalisation. The four metrics occupy incompatible numerical scales. RAM is measured in gigabytes; energy is measured in joules per hour; a raw difference of 1 GB in memory and 1 J in energy are not comparable without rescaling. Each metric is normalised to a Z-score prior to evaluation:

$$X_i^* = \frac{X_i - \bar{X}}{\sigma_X}$$

where $\bar{X}$ is the mean and σ_X the standard deviation computed across all candidate hosts in the mini-cluster at the time of the placement request. After normalisation, a unit deviation in CPU usage and a unit deviation in energy consumption carry equal weight in the fitness computation.

Weighted Euclidean distance. The aggregate score is computed as a weighted Euclidean distance over the four normalised resource gaps:

$$\text{fitness} = \sqrt{w_1 \cdot (\Delta C)^2 + w_2 \cdot (\Delta R)^2 + w_3 \cdot (\Delta IO)^2 + w_4 \cdot (\Delta E)^2}$$

where ΔX is the difference between the resource quantity requested by the incoming VM and the quantity available on the candidate host. Squaring the gaps serves a specific purpose: a host with 5% remaining CPU headroom receives a penalty that is an order of magnitude larger than one with 50% remaining, even if the absolute difference in the ΔC terms is modest. This non-linear penalisation is the mechanism by which the fitness function prioritises availability over aggregate utilisation.

Weight vector and default values. The weights (w_1, w_2, w_3, w_4) are real-valued parameters summing to unity:

$$\sum_{i=1}^{4} w_i = 1, \quad w_i \geq 0.$$

At initialisation, the weight vector is set to the uniform prior $w_i = 0.25\ \forall i$, expressing no prior knowledge about which resource dimension is the binding constraint. This baseline is chosen for its reproducibility: any deployment starts from the same configuration regardless of hardware heterogeneity. The weight vector is subsequently adjusted by the predictive model described in Section **??**; if the cluster enters a phase of sustained RAM pressure, w_2 increases and the remaining weights decrease proportionally, causing the algorithm to prioritise memory-rich hosts in subsequent placements. The application designer may also override the initial vector to encode domain knowledge --- for example, setting a higher w_4 for deployments where energy cost is a primary concern.

The algorithm seeks to *minimise* this score; the placement recommended is that which leaves the host with the largest, most balanced resource surplus across all four dimensions.

2.5.4 Reactive Scaling

Two complementary scaling modes are triggered by monitoring-layer alerts:

Horizontal scaling addresses scenarios in which request volumes exceed the processing capacity of the current workload instances. It is triggered when HTTP 5xx error rates surpass defined thresholds (1% at warning severity, 5% at critical severity), or when network packet loss exceeds 2%. The response involves instantiating a new, identical host. The WOA-DE algorithm determines its optimal physical placement, and the new instance is appended to the load-balancing pool to immediately absorb excess traffic [8].

Vertical scaling targets resource saturation intrinsic to a specific host. CPU utilisation exceeding 80% (warning) or 95% (critical) triggers the allocation of an additional virtual core. Memory utilisation exceeding 80% triggers a 20% capacity increase. When the current physical host lacks sufficient free capacity for vertical expansion, the orchestration layer initiates a live migration: the WOA-DE algorithm is invoked to identify an optimal destination host, and the workload is relocated without interruption [4].

Table 2.1 summarises the complete threshold matrix.

2.6 Containerisation and Workload Scheduling

The containerisation layer manages the execution of application workloads across the virtual machine fleet. While the WOA-DE hybrid algorithm governs the mapping of

Table 2.1: Reactive scaling threshold matrix.

Metric	Warning (%)	Critical (%)	Remediation
CPU utilisation	80	95	Add 1 virtual core; migrate if host saturated
RAM utilisation	60	80	Add 20% RAM; migrate if host saturated
Disk I/O free	<20	<10	Add 15 GB disk; migrate if host saturated
HTTP 5xx error rate	1	5	Horizontal scaling (new instance)
Network packet loss	---	2	Horizontal scaling (new instance)

virtual machines to physical hardware, workload-to-VM scheduling employs a deliberate **Best-Fit** heuristic. For each pending workload unit, the scheduler identifies the active virtual machine that possesses the minimum available capacity sufficient to satisfy the workload's resource request.

Best-Fit minimises resource fragmentation across the VM fleet [6]. By packing active virtual machines to near-maximum capacity, it maximises the fraction of VMs that are entirely idle. Idle instances can then be suspended or powered down, yielding reductions in energy consumption and licensing overhead [14]. The scheduler additionally applies affinity rules: workload units belonging to the same service group are preferentially placed within their designated mini-cluster, preserving network locality and limiting cross-boundary traffic.

2.7 Monitoring and Observability Architecture

2.7.1 Telemetry Collection

The monitoring subsystem employs a pull-based collection architecture: a central aggregation component periodically requests metric samples from agents deployed on every managed node [13]. The collection scope spans all four layers: physical server metrics (thermal, power, storage I/O, conforming to data-centre infrastructure monitoring standards [1]), hypervisor metrics such as CPU steal time and memory balloon activity, software-defined network throughput, and application-layer health endpoints.

Collected telemetry is subjected to time-windowed retention policies. High-resolution data is preserved during anomalous events; normal baseline periods are subject to progressive downsampling to bound storage growth [10].

2.7.2 Alert Management

The alert management subsystem translates raw metric time-series into actionable orchestration commands. When a metric breaches a defined threshold, the subsystem emits a structured alert payload. To prevent *alert storms* --- conditions in which a single physical failure triggers thousands of downstream alerts across dependent workloads --- the subsystem implements grouping and inhibition rules [11]. Related symptoms are consolidated into a single root-cause alert; only that consolidated alert is forwarded to the orchestration engine.

Table 2.2 classifies the principal categories of monitored metrics and their architectural purpose.

Table 2.2: Monitored metric categories and their observability purposes.

Category	Sources	Purpose
Hardware health	Physical hosts, power units	Detect hardware degradation; inform failure-domain isolation
CPU utilisation	All virtual hosts	Trigger vertical scaling and live migration
Memory utilisation	All virtual hosts	Trigger RAM expansion or migration
Disk I/O throughput	All virtual hosts	Detect I/O saturation; trigger disk expansion
Network throughput	Overlay network devices	Monitor inter-segment traffic; detect anomalous flows
Application errors	Workload health endpoints	Trigger horizontal scaling on elevated 5xx rates
Energy consumption	Physical hosts, workload agents	Inform energy-weighted placement decisions
Workload availability	Uptime probes	Detect unreachable instances; trigger failover

2.8 Intelligence and Security

2.8.1 Predictive Model and Dynamic Weight Recalibration

Fixed alert thresholds assume that the workload distribution on the cluster is stationary, an assumption that fails for microservices deployments subject to diurnal traffic cycles and application version changes. A gradient-boosted predictive model replaces this rigidity: it analyses the current metric snapshot across all monitored VMs and outputs both a bottleneck classification and a recalibrated weight vector for the placement fitness function.

The model's primary output is a revised (w_1, w_2, w_3, w_4) vector. If the cluster exhibits sustained RAM pressure --- a bottleneck that pure threshold logic may not detect until multiple VMs cross their warning levels simultaneously --- the model increases w_2 and reduces the others proportionally, shifting the placement algorithm's priorities before the next VM request arrives. Similarly, a period of elevated energy consumption pushes w_4 upward, directing the algorithm toward energy-efficient hosts.

The model also predicts dynamic alert thresholds for CPU, RAM, and HTTP error rates. These replace the static percentage values in Table 2.1 with context-sensitive values calibrated to the current baseline behaviour of the fleet.

Limitation on incremental learning. The predictive model is not natively trained in an online manner. Gradient-boosted tree models do not support parameter-level updates from individual observations the way neural networks do; online learning requires either appending new trees to the existing ensemble or retraining from a growing dataset. The current architecture uses batch re-inference with soft-label pseudo-targets derived from the model's own previous predictions to approximate adaptation. This approach carries a known risk of model collapse under distribution shift: if the pseudo-labels diverge from the true cluster state, the model may reinforce its own errors over successive batches. Ground-truth labels for supervised retraining --- derived from post-hoc resolution outcomes of detected alerts --- have not yet been collected; the incremental update mechanism is therefore disabled in the present implementation and is identified as a required engineering step before production deployment.

2.8.2 Dual-Agent Decision System

Safe autonomous remediation requires a separation between the agent that proposes actions and the agent that authorises them. A single agent combining both functions can rationalise unsafe actions to itself; an independent validator with no investment in the proposal cannot.

The **Planner** observes the cluster state by executing a mandatory sequence of read operations --- health summary, bottleneck prediction, recent action log, warning queue, node statistics, targeted resource detail --- before formulating a remediation proposal. The proposal is expressed as a structured record containing the intended tool, its parameters, the initiating risk class, an expected effect, and a rollback procedure. The Planner does not execute any write operation.

The **Critic** receives the proposal and the current cluster snapshot and evaluates it against a formal constraint set. Table **??** specifies the seven constraints, the trigger condition for each, the verdict issued when violated, and the alternative action suggested.

Table 2.3: Formal Critic constraint set.

#	Constraint	Trigger condition	Verdict	Alternative
1	Node headroom (CPU)	Target host CPU $> 85\%$ at time of evaluation	REJECT	`migrate_vm` to a host with CPU $< 80\%$
2	Node headroom (RAM)	Target host RAM $> 85\%$	REJECT	Clone instance; migrate if no eligible host
3	Cluster saturation	No host has CPU $< 80\%$	REJECT scale-up actions	Delete or suspend idle instances first
4	Duplicate action	Identical (tool, resource ID) pair in the action log within the last 5 minutes	REJECT	Wait for previous action to stabilise
5	Execution loop	Same (tool, resource ID) pair executed 3 or more times in 24 hours without resolution	REJECT; escalate to HIGH risk	Propose a fundamentally different remediation
6	Direction correctness	Scale-down proposal specifies equal or greater resource than current; scale-up specifies equal or fewer replicas	REJECT	Correct the direction; re-propose
7	Bottleneck alignment	Proposed action targets a resource dimension orthogonal to the detected bottleneck	FLAG (not reject); require explicit justification in reason field	Propose a resource-aligned action

The Critic operates under a *monotonic risk policy*: the risk classification of a proposal may be escalated but never reduced. Enforcement is structural: the final risk level is computed as the maximum of the static floor defined by the action type, the Planner's declared level, and the Critic's assessment. An adversarial prompt that attempts to reclassify a destructive action as low-risk is therefore defeated by the static floor, regardless of the reasoning presented.

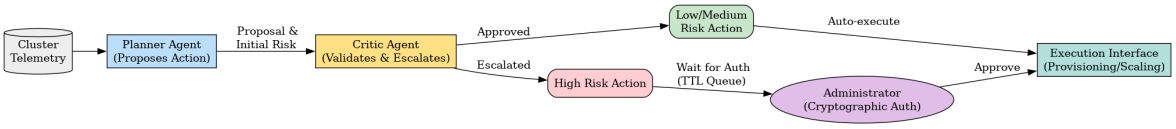


Figure 2.5: Dual-agent decision system: the Planner proposes, the Critic validates against the formal constraint set, and the execution layer acts only on approved proposals.

2.8.3 Risk-Based Escalation Matrix

The execution authority of the agent system is strictly governed by a three-tier escalation matrix, presented in Table 2.3.

Table 2.4: Risk-based escalation matrix for autonomous action execution.

Risk tier	Action types	Execution policy	Human required
Low	Add virtual CPUs, increase deployment replicas	Executed autonomously upon Critic approval; logged immediately	No
Medium	Reduce virtual resources, live migration, container restart, clone instance [8]	Executed autonomously upon Critic approval; operator notified by log	No
High	Power off workload, delete instance, delete deployment, drain physical node	Halted; queued in approval log; administrator notified; executed only upon explicit authorisation	Yes

2.8.4 Security Architecture

The integration of autonomous cognitive agents into an infrastructure management pipeline creates attack surfaces that are absent from manually administered clusters. An agent that can execute VM migrations, scale deployments, and stop running services is a powerful target: a successful adversarial injection into its reasoning context could

cause it to misclassify a destructive action as routine, to disable monitoring, or to drain a critical node under load. The security model is therefore structured around the specific threats introduced by the agent architecture, not around a generic checklist of security properties.

Threat: lateral movement from a compromised tenant workload. The three-segment overlay network with default-deny routing prevents a compromised container in the hospital or ministry segment from reaching the monitoring infrastructure or the orchestration API. A container that establishes an unexpected outbound connection hits the implicit deny rule before its packets reach the monitoring segment. The monitoring plane is thus reachable from the outside only through explicit permit rules, which are limited to the telemetry collection protocol.

Threat: forged remediation commands. All internal API calls within the orchestration pipeline require a shared API key validated at every endpoint. A forged proposal injected at any stage is rejected before execution. Telemetry collection traffic and command traffic are transmitted over encrypted channels, preventing passive observation of API key material by an attacker with access to the physical network.

Threat: adversarial prompt injection into agent reasoning. If a malicious actor embeds instructions inside a VM's hostname, metric labels, or log output that the Planner reads during its mandatory data collection sequence, those instructions could influence the proposed action. Two mechanisms limit this risk. First, the Critic evaluates the proposal independently against the formal constraint table (Table **??**); an injected instruction that leads the Planner to propose an unsafe action is caught by Constraint 1--7 before execution. Second, the monotonic risk floor prevents the injected content from lowering the risk classification of a destructive action below its static type-level floor.

Threat: privilege escalation through the agent's execution scope. The Planner and Critic agents hold no infrastructure credentials. They output structured proposals to a separate execution layer, which performs secondary validation and issues the actual API call. An agent that is manipulated to propose an unusually broad action still cannot execute it without passing through the execution layer's validation.

Audit and forensic traceability. Every proposal, Critic verdict, execution outcome, and human approval is appended to an immutable log [11, 7]. The append-only property ensures that a compromised agent cannot remove evidence of actions it has taken. Post-incident forensic analysis can therefore reconstruct the full decision chain leading to any infrastructure change.

Workload hardening. Virtual machine images are built with minimal driver sets to reduce the kernel attack surface. Container workloads run with a restricted capability set; persistent write access to the filesystem is granted only where the application explicitly requires it. Scheduling affinity rules separate workloads from different security domains onto distinct physical hosts, limiting the blast radius of a kernel-level side-channel attack [7].

2.9 Conclusion

The four design gaps identified at the close of Chapter 1 are each addressed by a specific component of the architecture developed here. The WOA-DE hybrid algorithm resolves the premature convergence problem of single-paradigm bio-inspired optimisers through population splitting: one subset exploits the current best placement via WOA's spiral geometry while the other maintains diversity through DE's algebraic mutation. The multi-objective fitness function with Z-score normalisation and quadratic penalisation integrates CPU, RAM, I/O, and energy into a single comparable score, sidestepping the unit-incommensurability problem that causes single-objective heuristics to produce systematically skewed placements. The XGBoost prediction layer closes the loop between monitoring and placement by recalibrating the fitness weight vector in response to observed bottleneck patterns, replacing the static weight assignment that all prior work assumes. Finally, the Planner/Critic system with its formal seven-constraint validation table and monotonic risk floor provides autonomous remediation authority bounded by explicit, auditable governance rules --- a requirement that the reviewed literature does not address.

What this chapter does not provide is an empirical evaluation of these mechanisms. Performance claims --- that the WOA-DE hybrid converges faster than pure WOA on a placement problem of a given dimension, or that the weight recalibration loop reduces mean time to recovery --- remain to be measured. Chapter 3 describes the implementation and identifies the experiments required.

Chapter 3

Implementation

3.1 Introduction

Chapter 2 described what the infrastructure must do and why each design decision was made. This chapter describes how those decisions were realised, which technologies were selected, and what the resulting system looks like at the configuration level. The exposition is grounded directly in the project artefacts: the source code, the network router configuration, the monitoring rule files, and the service deployment descriptors. Where these artefacts do not yet cover a section --- because the physical deployment was not finalised at the time of writing --- the gap is marked explicitly with [TO BE CONFIRMED] and must be completed before submission.

3.2 Technology Selection and Justification

For each architectural layer defined in Chapter 2, a specific technology was selected on the basis of licence model, community maturity, on-premises suitability, and performance characteristics. Table 3.1 summarises these choices.

3.3 Implementation Environment

The proof-of-concept cluster is deployed on-premises at the USTHB Faculty of Informatics. It consists of physical servers connected via a managed switching fabric, administered through the Proxmox VE hypervisor management interface.

The cluster encompasses eight physical nodes, identified as `pfe0` through `pfe7`. Nodes are partitioned into two primary roles: Kubernetes worker nodes hosting application and monitoring workloads, and dedicated Kubernetes master nodes forming the control plane. A subset of nodes additionally hosts LXC containers for lightweight infrastructure services. Detailed CPU, RAM, and storage specifications are to be confirmed from the Proxmox Datacenter inventory following finalisation of the hardware allocation.

Table 3.1: Technology selection by architectural layer.

Architectural layer	**Technology**	**Version**	**Justification**
Virtualisation	Proxmox VE	[TO BE CONFIRMED]	Open-source (AGPL), integrates KVM and LXC natively, REST API, HA and live migration without vendor cost
Container orchestration	Kubernetes	[TO BE CONFIRMED]	CNCF-graduated; rich scheduler primitives, namespace isolation, HPA, NetworkPolicy; dominant community
Monitoring	Prometheus	[TO BE CONFIRMED]	Pull-based scraping, PromQL, HTTP SD; native Kubernetes integration; CNCF flagship project
Visualisation	Grafana	[TO BE CONFIRMED]	Open-source dashboarding; native Prometheus datasource; alerting integration
Virtual routing	OPNsense	[TO BE CONFIRMED]	Open-source BSD firewall; per-interface VLAN routing, stateful packet inspection, NAT, REST API
AI agent framework	Agno (Python)	[TO BE CONFIRMED]	Lightweight multi-agent library; tool-call abstraction; SQLite-backed agent memory
Prediction model	XGBoost	[TO BE CONFIRMED]	Gradient boosting; low inference latency; interpretable feature importances; established in cluster resource prediction literature; ensemble extension supports batch re-inference for weight recalibration
LLM inference	Groq Cloud API	---	Sub-second LLM inference for Planner/Critic reasoning (llama-3.3-70b-versatile)
Service discovery API	FastAPI (Python)	[TO BE CONFIRMED]	Asynchronous HTTP; Pydantic validation; native OpenAPI documentation

Table 3.2: Physical host inventory. All values are to be confirmed from the Proxmox `Datacenter` view.

Host ID	Role	CPU	RAM	Storage	OS
pfe0	[TO BE CONFIRMED]	[TBC]	[TBC]	[TBC]	[TBC]
pfe1	[TO BE CONFIRMED]	[TBC]	[TBC]	[TBC]	[TBC]
pfe2	[TO BE CONFIRMED]	[TBC]	[TBC]	[TBC]	[TBC]
pfe3	[TO BE CONFIRMED]	[TBC]	[TBC]	[TBC]	[TBC]
pfe4	[TO BE CONFIRMED]	[TBC]	[TBC]	[TBC]	[TBC]
pfe5	[TO BE CONFIRMED]	[TBC]	[TBC]	[TBC]	[TBC]
pfe6	[TO BE CONFIRMED]	[TBC]	[TBC]	[TBC]	[TBC]
pfe7	[TO BE CONFIRMED]	[TBC]	[TBC]	[TBC]	[TBC]

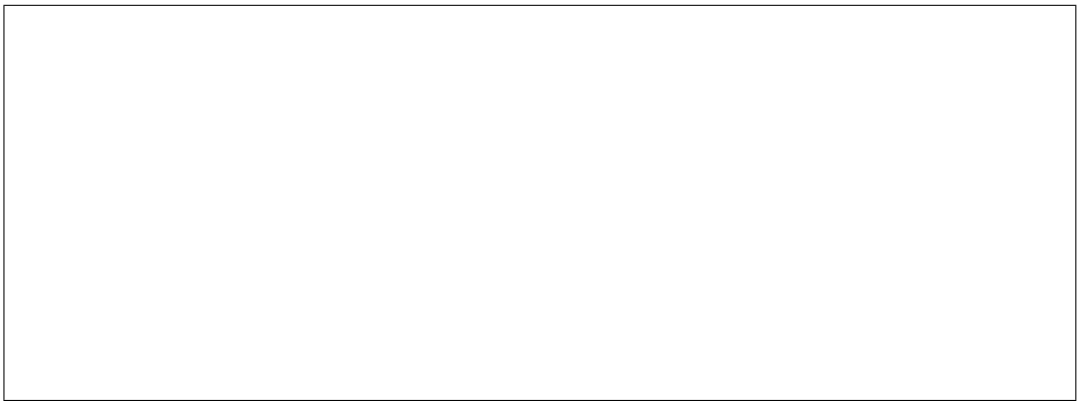


Figure 3.1: [PLACEHOLDER --- Screenshot of: Proxmox VE Datacenter node summary view, listing pfe0 through pfe7 with CPU and memory utilisation bars.]

3.4 Virtualisation with Proxmox VE

Proxmox VE implements the virtualisation layer defined in Chapter 2, providing KVM-based virtual machine management and LXC container support through a unified web interface and REST API. The platform supports live migration between cluster nodes, shared storage pools, and high-availability VM groups, directly instantiating the failure-domain redundancy model described in Section 2.4.

Virtual machines are created via the Proxmox API or web interface with explicit resource envelopes (vCPU count, RAM, disk size). Resource pools group VMs by functional domain, and HA groups ensure that critical VMs are automatically restarted on surviving nodes following a host failure. The WOA-DE placement algorithm interfaces with the Proxmox REST API to query real-time node utilisation metrics and to submit live migration commands when the fitness function determines that relocation is warranted.

Table 3.3: VM inventory. All values to be confirmed from the Proxmox `pvenode list --full` output.

VM Name	vCPU	RAM	Disk	Hosted service	Mini-cluster
		TO BE CONFIRMED			
	[TBC]	[TBC]	[TBC]	[TBC]	[TBC]
(Repeat rows from Proxmox VM list)					

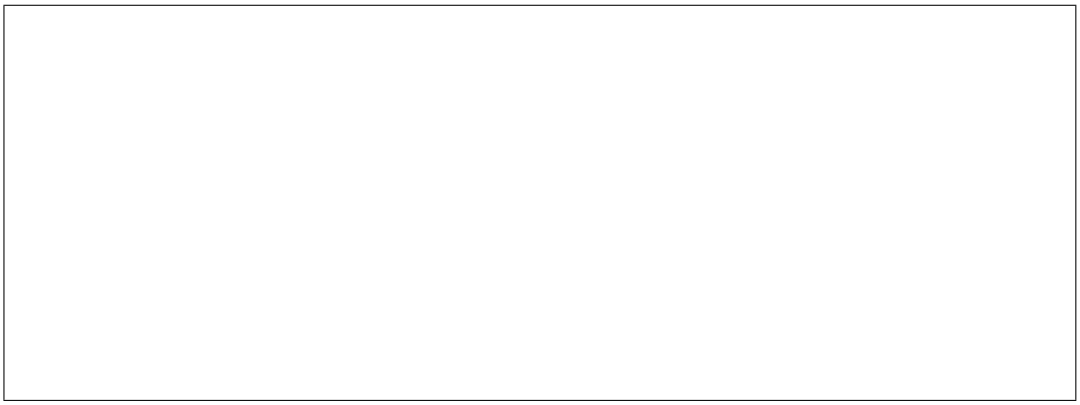


Figure 3.2: [PLACEHOLDER --- Screenshot of: Proxmox VE web dashboard displaying the VM summary table for the cluster with resource allocation and node placement.]

3.5 Container Orchestration with Kubernetes

Kubernetes implements the containerisation layer from Chapter 2. Each architectural concept from that layer maps to a specific Kubernetes primitive:

The mini-cluster partitioning concept corresponds to Kubernetes **Namespaces**. Each mini-cluster is realised as a distinct namespace, providing logical isolation, independent resource quotas, and a scope for NetworkPolicy enforcement. Table 3.4 lists the five namespaces instantiated for the proof-of-concept deployment.

Container scheduling within a mini-cluster corresponds to the **Kubernetes Scheduler** operating under Pod resource requests and limits. The Best-Fit heuristic is approximated by setting appropriate resource requests so the scheduler assigns Pods to the most-occupied feasible node.

Horizontal scaling corresponds to **HorizontalPodAutoscaler** objects, which increase replica counts when CPU or custom metrics exceed defined targets.

Service discovery corresponds to Kubernetes **Services** with cluster-internal DNS resolution, allowing each microservice to reference its dependencies by name rather than by IP address.

Figure 3.3: [PLACEHOLDER --- Screenshot of: `kubectl get nodes -o wide` showing all cluster nodes with their roles and Kubernetes version.]

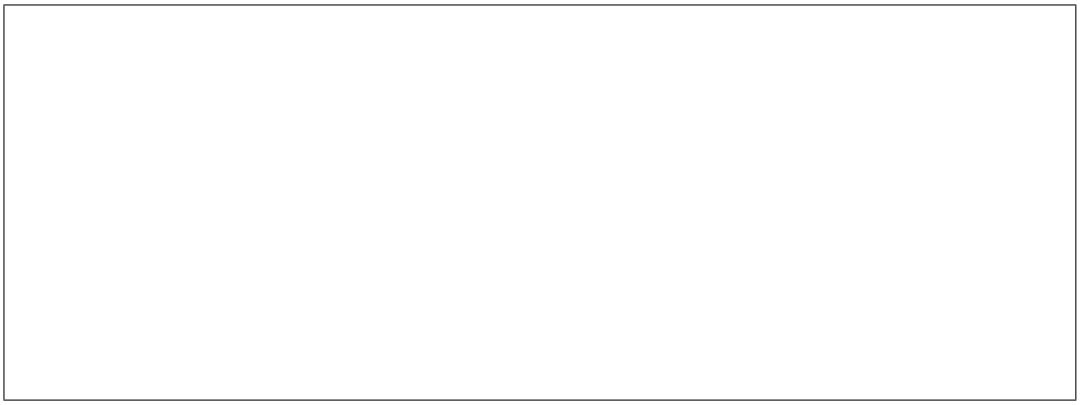


Figure 3.4: [PLACEHOLDER --- Screenshot of: `kubectl get pods --all-namespaces` showing all running pods across the five mini-cluster namespaces.]

Table 3.4: Kubernetes object inventory (partial --- to be completed with `kubectl get all --all-namespaces` output).

Kind	Name	Namespace	Purpose
Namespace	mini-cluster-1	---	Authentication, blockchain, crypto, messaging
Namespace	mini-cluster-2	---	Clinical care services (hospital)
Namespace	mini-cluster-3	---	Hospital and patient management
Namespace	mini-cluster-4	---	Ministry-tier services and frontend
Namespace	mini-cluster-5	---	Facility management and hospital frontend
Deployment	auth	mini-cluster-1	[TO BE CONFIRMED]
Deployment	blockchain-service	mini-cluster-1	[TO BE CONFIRMED]
Deployment	cpabe-service	mini-cluster-1	[TO BE CONFIRMED]
Deployment	rabbitmq	mini-cluster-1	Asynchronous message broker
Deployment	soins	mini-cluster-2	[TO BE CONFIRMED]
Deployment	rendez-vous	mini-cluster-2	[TO BE CONFIRMED]
Deployment	consultations	mini-cluster-2	[TO BE CONFIRMED]
Deployment	prescriptions-bilans-radios	mini-cluster-2	[TO BE CONFIRMED]
Deployment	interventions	mini-cluster-2	[TO BE CONFIRMED]
Deployment	hopitaux	mini-cluster-3	[TO BE CONFIRMED]
Deployment	hospitalisations	mini-cluster-3	[TO BE CONFIRMED]
Deployment	patients	mini-cluster-3	[TO BE CONFIRMED]
Deployment	personnel	mini-cluster-3	[TO BE CONFIRMED]
Deployment	notifications	mini-cluster-4	[TO BE CONFIRMED]
Deployment	codifications	mini-cluster-4	Medical code reference (3 replicas)
Deployment	frontend-ministry	mini-cluster-4	Ministry user interface
Deployment	chambres-lits	mini-cluster-5	[TO BE CONFIRMED]
Deployment	reception	mini-cluster-5	[TO BE CONFIRMED]
Deployment	pharmacy	mini-cluster-5	[TO BE CONFIRMED]
Deployment	frontend-hospital	mini-cluster-5	Hospital user interface

Service, ConfigMap, PersistentVolumeClaim, and HPA objects: [TO BE CONFIRMED from kubectl output]

3.6 Deployed Application --- Medical Information System

3.6.1 Application Overview

The proof-of-concept workload is a distributed medical information system designed to serve both a national ministry-tier administrative layer and a hospital-tier operational layer. The system follows a microservices architecture, deployed as a set of containerised services across five Kubernetes namespaces corresponding to the five mini-clusters defined in Chapter 2.

The system is divided into two functional groups. The **Ministry group** encompasses services that operate at the national scale: identity and authentication management, cryptographic access-control policy enforcement (CP-ABE), distributed ledger record-keeping, medical code reference data, inter-institutional notifications, and the ministry-facing web frontend. The **Hospital group** encompasses services that operate at the institutional scale: patient admission and record management, appointment scheduling, medical consultations and clinical notes, prescription and laboratory result management, radiology workflows, surgical interventions, ward and bed management, pharmacy dispensing, staff administration, and the hospital-facing web frontend.

Inter-service communication relies on two mechanisms. Synchronous, request-response interactions use HTTP REST APIs exposed on each service's declared port. Asynchronous, event-driven interactions use the RabbitMQ message broker deployed in `mini-cluster-1`, decoupling producers from consumers and enabling fan-out notification patterns.

3.6.2 Microservice Catalogue

Table 3.5 lists all deployed microservices. Language, framework, and port assignments are to be confirmed from the final container images; the information below reflects the available deployment descriptors.

Table 3.5: Microservice catalogue of the medical information system.

Service name	Group	Framework	Port	Role
auth	Ministry / Shared	[TO BE CONFIRMED]	[TBC]	Authentication and session management

Table 3.5 continued…

Service name	Group	Framework	Port	Role
blockchain-service	Ministry / Shared	[TO BE CONFIRMED]	[TBC]	Distributed ledger for tamper-proof record provenance
cpabe-service	Ministry / Shared	[TO BE CONFIRMED]	[TBC]	Ciphertext-policy attribute-based encryption access control
rabbitmq	Shared (infra)	RabbitMQ	5672 / 15672	Asynchronous message broker
notifications	Ministry	[TO BE CONFIRMED]	[TBC]	Inter-institutional and user notification dispatch
codifications	Ministry	[TO BE CONFIRMED]	[TBC]	Medical classification codes reference (3 replicas)
frontend-ministry	Ministry	[TO BE CONFIRMED]	[TBC]	Ministry administrative web interface
soins	Hospital	[TO BE CONFIRMED]	[TBC]	Care act recording and management
rendez-vous	Hospital	[TO BE CONFIRMED]	[TBC]	Appointment scheduling
consultations	Hospital	[TO BE CONFIRMED]	[TBC]	Medical consultation records
prescriptions-bilans-radios	Hospital	[TO BE CONFIRMED]	[TBC]	Prescriptions, laboratory results, radiology orders
interventions	Hospital	[TO BE CONFIRMED]	[TBC]	Surgical intervention tracking

Table 3.5 continued…

Service name	Group	Framework	Port	Role
hopitaux	Hospital	[TO BE CONFIRMED]	[TBC]	Hospital registry and metadata management
hospitalisations	Hospital	[TO BE CONFIRMED]	[TBC]	Inpatient admission workflows
patients	Hospital	[TO BE CONFIRMED]	[TBC]	Patient identity and demographic records
personnel	Hospital	[TO BE CONFIRMED]	[TBC]	Medical and administrative staff management
chambres-lits	Hospital	[TO BE CONFIRMED]	[TBC]	Ward, room, and bed occupancy management
reception	Hospital	[TO BE CONFIRMED]	[TBC]	Patient intake and registration desk
pharmacy	Hospital	[TO BE CONFIRMED]	[TBC]	Dispensing records and stock management
frontend-hospital	Hospital	[TO BE CONFIRMED]	[TBC]	Hospital operational web interface

3.6.3 Mini-Cluster to Application Group Mapping

The five mini-clusters correspond directly to the two application groups: `mini-cluster-1` hosts the shared infrastructure services (authentication, cryptographic access control, blockchain, messaging) used by both groups; `mini-cluster-2` and `mini-cluster-3` host hospital clinical and management services respectively; `mini-cluster-4` hosts the ministry-tier services and frontend; and `mini-cluster-5` hosts facility-management and the hospital frontend. Ministry and hospital workloads occupy distinct Kubernetes namespaces and consequently distinct network isolation boundaries, reflecting the three-segment overlay model designed in Chapter 2.

Figure 3.5: [PLACEHOLDER --- Application architecture diagram showing the five mini-clusters, the services within each, and inter-service communication paths via RabbitMQ and REST.]

Figure 3.6: [PLACEHOLDER --- Screenshot of: the application frontend running in a browser, showing the ministry or hospital user interface.]

3.7 Monitoring Implementation

3.7.1 Prometheus Service Discovery

Prometheus is configured to use HTTP-based service discovery (HTTP SD), allowing monitoring targets to register and deregister dynamically without restarting the Prometheus process. A dedicated FastAPI service exposes four target list endpoints, one per exporter type:

- `/targets/node_exporter` --- virtual machine operating system metrics
- `/targets/scaphandre` --- per-VM energy consumption metrics
- `/targets/snmp_exporter` --- network device interface metrics
- `/targets/blackbox_exporter` --- application endpoint reachability probes

The service discovery API enforces API-key authentication on write operations (target registration and deregistration) while leaving read endpoints openly accessible to the Prometheus scraper. Target registrations are persisted to a JSON-backed database, providing idempotent re-registration semantics.

3.7.2 Scrape Targets

Table 3.6 lists the four Prometheus scrape jobs as defined in `prometheus.yml`.

Table 3.6: Prometheus scrape job configuration.

Job name	Port	Discovery method	Metrics scope
`prometheus`	9090	Static (localhost)	Prometheus self-metrics
`node_exporter`	9100	HTTP SD (5s refresh)	CPU, RAM, disk, network per host
`scaphandre`	8080	HTTP SD (5s refresh)	Per-VM energy (microwatts)
`snmp`	9116	HTTP SD (5s refresh)	Network device interface counters
`blackbox-exporter`	9115	HTTP SD (5s refresh)	HTTP endpoint reachability probes

The `scaphandre` job uses metric relabelling to extract the VM identifier and name from the process command line, attaching them as `vm_id` and `vm_name` labels. This enables per-VM energy attribution in Prometheus queries. The `snmp` and `blackbox-exporter` jobs use standard Prometheus relabelling to pass the target address as a query parameter to the respective exporter.

3.7.3 Alert Rules

The alert rules defined in `vm_scaling.yml` implement the threshold matrix from Section 2.5.3. Table 3.7 summarises each rule group.

Table 3.7: Prometheus alert rules and their corresponding remediation actions.

Alert name	Threshold	Duration	Declared remediation
`HTTP5xxWarning`	>1%	2 min	Horizontal scaling
`HTTP5xxCritical`	>5%	1 min	Horizontal scaling
`PacketLossCritical`	>2%	1 min	Horizontal scaling
`CPUWarning`	>80%	2 min	Vertical scaling (CPU); migrate if PM insufficient
`CPUCritical`	>95%	1 min	Add 1 vCPU; migrate if PM insufficient
`RAMWarning`	>60%	2 min	Vertical scaling (RAM); migrate if PM insufficient
`RAMCritical`	>80%	2 min	Add 20% RAM; migrate if PM insufficient
`DiskIOWarning`	Free <20%	5 min	Monitor
`DiskIOCritical`	Free <10%	2 min	Add 15 GB disk; migrate if PM insufficient
`VMUnreachable`	up = 0	2 min	Check network, SSH, Proxmox host status
`ScaphandreDown`	up = 0	1 min	Restart Scaphandre service
`VMHighLatency`	Scrape >2 s	3 min	Investigate network
`VMRebootDetected`	Uptime <5 min	1 min	Monitor stability; inspect journal
`VMOOMKilled`	OOM kills >0	1 min	Increase RAM or terminate memory-intensive processes
`VMDiskAlmostFull`	Available <2%	1 min	Emergency disk cleanup
`VMHighPowerConsumption`	>50 W	5 min	Investigate power anomaly

Telemetry is forwarded to an InfluxDB instance via Prometheus remote write, enabling long-term storage and historical analysis independent of Prometheus's local retention window. The remote write endpoint is secured with a bearer token stored as a file at a path outside the Prometheus configuration file, in conformance with the principle of secret externalisation.

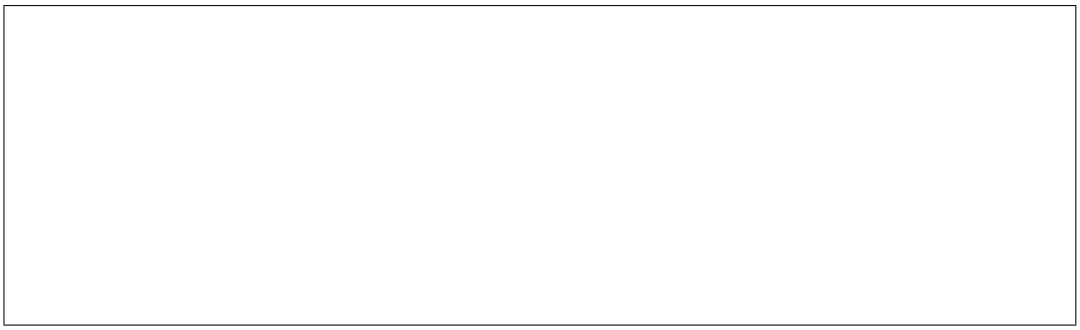


Figure 3.7: [PLACEHOLDER --- Screenshot of: Prometheus Targets page showing all scrape jobs with their UP/DOWN status and last scrape duration.]

Figure 3.8: [PLACEHOLDER --- Screenshot of: Grafana dashboard displaying CPU, RAM, and energy metrics for the cluster.]

3.8 Network Implementation

3.8.1 OPNsense Virtual Router

The overlay network is implemented by an OPNsense virtual router deployed as a dedicated virtual machine within the cluster. OPNsense provides the stateful firewall, NAT, and inter-segment routing functions described in Section 2.4. Three logical network interfaces are configured on the virtual router, corresponding to the three segments of the isolation model:

- **Hospital segment**: subnet `10.10.10.0/24`, gateway `10.10.10.1`. Hosts all hospital-tier Kubernetes workloads.
- **Ministry segment**: subnet `10.20.20.0/24`, gateway `10.20.20.1`. Hosts all ministry-tier Kubernetes workloads.
- **Monitoring segment**: subnet `10.30.30.0/24`, gateway `10.30.30.1`. Hosts Prometheus, Grafana, InfluxDB, and the service discovery API.

An additional WAN interface connects the cluster to the campus network (`172.25.5.0/24`), providing the upstream route for external client traffic and administrative access.

3.8.2 Firewall Rules

The OPNsense ruleset enforces the default-deny posture defined in Chapter 2. Explicit permit rules cover the following traffic flows, derived from the deployed configuration:

- Inbound HTTP and HTTPS from the campus network through the NGINX load-balancer VM to the VXLAN-connected Kubernetes service endpoints.
- ICMP reachability probes from the NGINX VM to VXLAN targets.
- SSH and ICMP from the LAN (campus) to cluster VMs for administrative access.
- Prometheus scrape traffic from the monitoring segment to all other segments (`prometheus rule`).
- Load-balancer to VXLAN HTTP and HTTPS traffic.
- Kubernetes API server access (`kube port`).
- Return traffic from the Kubernetes segments to the load balancer.

All other inbound and cross-segment traffic is implicitly rejected. NAT masquerades outbound traffic from all three segments behind the WAN interface address, preventing internal addressing from leaking to external networks.

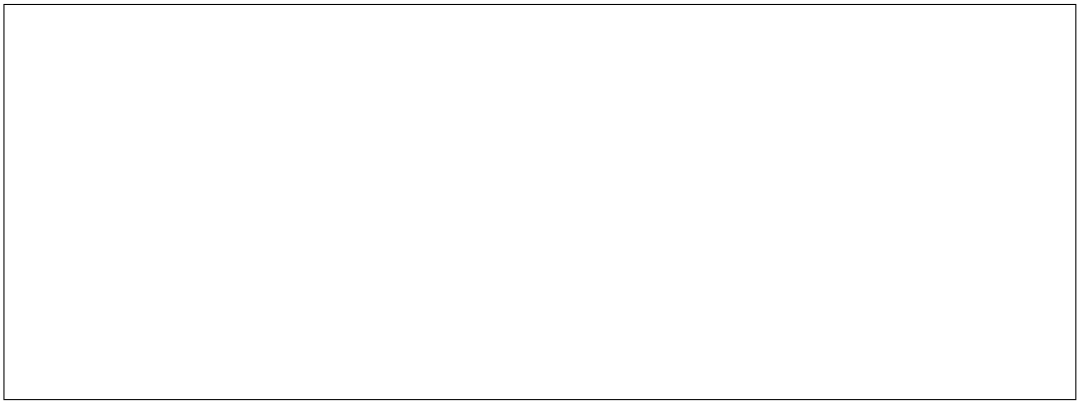


Figure 3.9: [PLACEHOLDER --- Network topology diagram showing the three VLANs (10.10.10.0/24, 10.20.20.0/24, 10.30.30.0/24), the OPNsense router, the NGINX load-balancer VM, and the campus WAN uplink.]

3.9 Autonomous Orchestration Implementation

3.9.1 Planner and Critic Agents

The dual-agent system described in Chapter 2 is realised using the Agno multi-agent Python library. Both the Planner and the Critic are instantiated as Agno `Agent`

objects, backed by a large language model served via the Groq Cloud inference API (`llama-3.3-70b-versatile`). The Planner is equipped with a comprehensive tool set (see Section 3.9.2) that allows it to read cluster state before proposing an action. The Critic receives no tools; it reasons purely over the structured JSON proposal and the cluster snapshot passed to it in its prompt context.

The Planner is configured with a SQLite-backed memory store that injects the last ten conversation turns into each new reasoning cycle, preserving situational awareness across invocations. A mandatory seven-step data collection sequence (cluster health, XGBoost prediction, action log, warning queue, node list, targeted VM/deployment inspection, VM action history) is enforced by the system prompt before any action proposal is permitted.

3.9.2 Tool Catalogue

The orchestration tools available to the Planner are classified by risk level in Table 3.8.

3.9.3 Email Approval Workflow

High-risk actions are not executed directly. Instead, the agent service serialises the proposed action, its Critic rationale, and a current cluster state summary into a structured plain-text email dispatched via SMTP to the designated administrator address. The email body includes two hyperlinks --- one to approve the action and one to reject it --- pointing to the agent service's `/approve/{action_id}` and `/reject/{action_id}` HTTP endpoints. The action record is stored in a SQLite approval queue table. If no response is received within 24 hours, the action is automatically rejected and logged. Upon approval, the agent service executes the corresponding API call against the cluster management API; upon rejection, it logs the decision with the administrator's rejection rationale if provided.

3.9.4 XGBoost Predictive Model

The machine learning service is implemented as a standalone FastAPI application that exposes a `/sync` endpoint. On each invocation, it accepts a batch of VM telemetry snapshots (CPU, RAM, I/O, HTTP error rates, network drop rates, energy consumption, and VLAN encoding), constructs a feature matrix, and runs inference across seven XGBoost models: four predicting the fitness weights ($w_{\text{cpu}}, w_{\text{ram}}, w_{\text{io}}, w_{\text{energy}}$) and three predicting dynamic alert thresholds (CPU critical, RAM critical, HTTP critical). The weight predictions are averaged across healthy (up = 1) VMs in the batch and normalised to sum to unity, yielding the recalibrated weight vector forwarded to the placement algorithm. Incremental online learning is currently suspended pending the

Table 3.8: Planner tool catalogue with risk classification.

Tool	Risk	Description
Read tools (no infrastructure modification)		
`get_health`	---	Cluster overview: node, VM, LXC counts; queue length
`list_nodes`	---	All Proxmox nodes with CPU, RAM, energy utilisation
`get_node`	---	Detailed stats for a single node
`list_vms / get_vm`	---	VM inventory and per-VM detailed utilisation
`list_lxc / get_lxc`	---	LXC container inventory and stats
`list_deployments`	---	Kubernetes deployment replica and limit summary
`get_deployment`	---	Single deployment detailed status
`get_warning_queue`	---	Active DE-WOA provisioning queue
`get_history_stats`	---	Historical resource gap mean and standard deviation
`get_xgboost_prediction`	---	Current bottleneck class, weights, confidence
`get_action_log`	---	Last 100 automated actions
`get_vm_action_history`	---	Per-VM action history (loop detection)
Write tools (execute after Critic approval)		
`scale_vm`	Low	Add vCPUs or RAM to running VM
`scale_up_deployment`	Low	Increase Kubernetes deployment replicas
`scale_down_vm`	Medium	Reduce vCPUs or RAM (OOM risk)
`clone_vm / clone_lxc`	Medium	Clone VM or LXC for horizontal scaling
`scale_down_deployment`	Medium	Decrease Kubernetes deployment replicas
`restart_vm / restart_lxc`	Medium	Reboot VM or LXC
`migrate_vm`	Medium	Live-migrate VM to a different Proxmox node
High-risk tools (deferred to email approval queue)		
`stop_vm / stop_lxc`	High	Graceful shutdown
`delete_vm / delete_lxc`	High	Permanent deletion (irreversible)
`delete_deployment`	High	Terminate all pods of a K8s deployment
`drain_node`	High	Mark Proxmox node for maintenance

availability of ground-truth labels, in conformance with a model-collapse prevention policy recorded in the codebase.

3.10 Results and Validation

3.10.1 Validation Conditions Status

The four proof-of-concept validation conditions defined in Section 2.2 are summarised below. Their confirmed status is to be updated following final deployment.

Table 3.9: Proof-of-concept validation status.

Condition	**Status**	**Evidence / Notes**
Automated VM provisioning without manual intervention	[TO BE CONFIRMED]	Tested via Proxmox API and WOA-DE algorithm invocation
Load anomaly detection and remediation within time budget	[TO BE CONFIRMED]	Prometheus alert + Planner/Critic cycle latency to be measured
Container scheduling without limit violations	[TO BE CONFIRMED]	Verified via `kubectl describe pod` resource section
Continuous reachability during scaling operations	[TO BE CONFIRMED]	HTTP probe continuity during synthetic load test

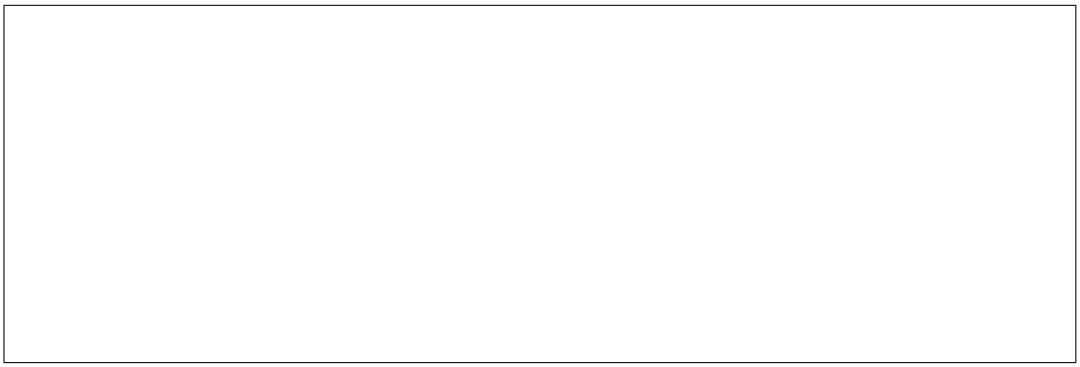


Figure 3.10: [PLACEHOLDER --- Screenshot of: agent action log showing a completed Planner--Critic--execution cycle, including the proposed action JSON, Critic verdict, and execution result.]

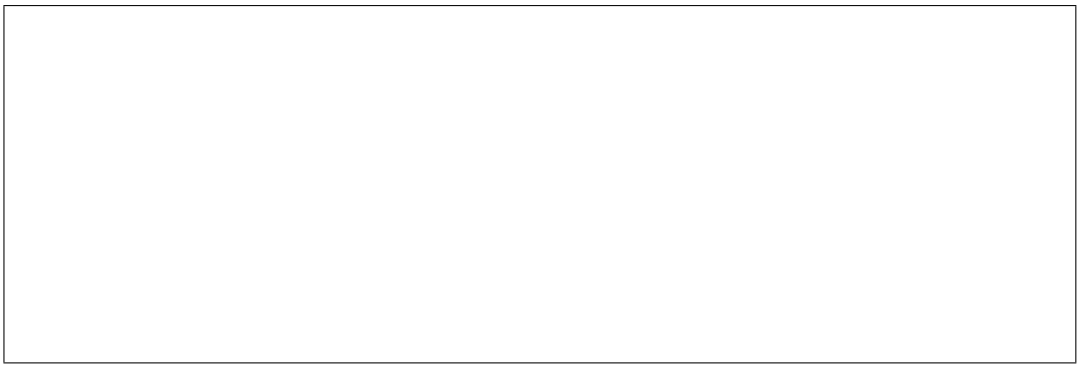


Figure 3.11: [PLACEHOLDER --- Screenshot of: WOA-DE placement output log or Proxmox live migration event confirming a successful VM relocation triggered by the autonomous agent.]

3.11 Conclusion

3.12 Conclusion

The architecture of Chapter 2 maps unambiguously onto the technology stack selected here. Proxmox VE provides VM lifecycle management, live migration, and the REST API through which the WOA-DE placement algorithm queries node metrics and issues migration commands. Kubernetes' five-namespace structure realises the mini-cluster partitioning, with NetworkPolicy objects enforcing the inter-namespace isolation rules from Section 2.4. The twenty microservices of the medical information system, spanning ministry and hospital functional groups, are distributed across those namespaces with inter-service communication handled by synchronous HTTP calls and the RabbitMQ broker. Prometheus collects telemetry from four exporter families, evaluates sixteen alert rules against the thresholds derived from the conceptual model, and forwards long-term data to InfluxDB. The OPNsense firewall enforces the three trust-domain network isolation with the specific rule set documented in Section 3.8. The dual-agent orchestration layer closes the control loop.

The quantitative validation required to substantiate the performance claims of Chapter 2 --- convergence speed, mean time to remediation, availability under simulated failure --- remains to be conducted. The experiments to be run, the metrics to be recorded, and the expected outcomes based on the design rationale are documented in Section 3.9, pending completion of the deployment environment.

General Conclusion

The problem posed at the outset --- how to deploy a high-load distributed application on an on-premises cluster while holding resource efficiency and service availability in simultaneous tension --- does not admit a solution at any single abstraction level. The research reported in this thesis therefore operates at three levels simultaneously: hardware partitioning, algorithmic placement, and autonomous governance.

At the hardware and network level, the four-layer architecture with explicit failure domains and a three-segment overlay network establishes the isolation boundaries that later layers depend on. Without fault-domain awareness at the physical layer, the placement algorithm has no way to distribute replicas meaningfully; without network isolation, the monitoring plane is as vulnerable as the tenant workloads it observes.

At the algorithmic level, the hybrid WOA-DE placement algorithm and its accompanying fitness function address three weaknesses identified in the reviewed literature: the premature convergence of single-paradigm bio-inspired optimisers, the incommensurability of heterogeneous resource metrics under naive aggregation, and the rigidity of static fitness weights in the face of shifting workload demands. The Z-score normalisation provides a principled solution to the first two. The connection between the XGBoost prediction layer and the weight vector provides a mechanism for the third, though the incremental retraining loop required to keep the prediction model itself current remains an open engineering problem pending the collection of ground-truth alert-resolution labels.

At the governance level, the Planner/Critic dual-agent system with its seven-constraint validation table and monotonic risk floor provides the first formally specified autonomous remediation framework in this line of work. Prior systems in the literature demonstrate automated scaling and placement but do not address the question of how to bound an agent's authority when its decisions are consequential and potentially irreversible.

Three immediate extensions follow from the limitations identified in this thesis. First, the ground-truth labelling framework for XGBoost retraining must be built: the most tractable approach is to label historical alert-resolution pairs using the outcome field of the action log, then retrain offline before re-deploying the model. Second, the static mini-cluster partitioning should be evaluated against a dynamic graph-clustering baseline to quantify the scheduling-efficiency loss introduced by boundary errors. Third, the Critic's constraint table is currently implemented as heuristic rules encoded in a language model system prompt; replacing at least the numerical con-

straints (headroom thresholds, duplication windows) with hard-coded checks in the execution layer would make the safety guarantees independent of prompt sensitivity.

The proof-of-concept deployment on the medical information system at USTHB provides the experimental context for all three extensions. Its completion is the immediate next step.

Bibliography

[1] G. Balabanov and N. S. Apostolov, ``Data Centre Infrastructure Monitoring,'' in *60th International Scientific Conference on Information, Communication and Energy Systems and Technologies (ICEST)*, IEEE, 2025.

[2] S. Mirjalili and A. Lewis, ``The Whale Optimisation Algorithm,'' *Advances in Engineering Software*, vol. 95, pp. 51--67, 2016.

[3] R. Storn and K. Price, ``Differential Evolution --- A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces,'' *Journal of Global Optimization*, vol. 11, no. 4, pp. 341--359, 1997.

[4] X. Lai, X. Zheng, and W. Lu, ``Energy-aware Virtual Machine Migration based on Dual Thresholds of Server Resource Utilization Rate considering Performance Interference,'' in *2025 IEEE 2nd International Conference on Big Data Science and Engineering (ICBDSE)*, IEEE, 2025.

[5] P. Zhang, M. Zhou, and X. Wang, ``An Intelligent Optimization Method for Optimal Virtual Machine Allocation in Cloud Data Centers,'' *IEEE Transactions on Automation Science and Engineering*, vol. 17, no. 4, pp. 1725--1735, 2020.

[6] R. Begam, W. Wang, and D. Zhu, ``TIMER-Cloud: Time-Sensitive VM Provisioning in Resource-Constrained Clouds,'' *IEEE Transactions on Cloud Computing*, vol. 8, no. 1, pp. 297--311, 2020.

[7] D. Saxena, I. Gupta, J. Kumar, A. K. Singh, and X. Wen, ``A Secure and Multiobjective Virtual Machine Placement Framework for Cloud Data Center,'' *IEEE Systems Journal*, vol. 16, no. 2, pp. 3163--3174, 2022.

[8] J. Sung, S. Han, and J. Kim, ``Cloning-based virtual machine pre-provisioning for resource-constrained edge cloud server,'' *Cluster Computing*, vol. 27, pp. 1831--1847, 2024.

[9] K. Senjab, S. Abbas, N. Ahmed, and A. R. Khan, ``A survey of Kubernetes scheduling algorithms,'' *Journal of Cloud Computing*, vol. 12, no. 87, 2023.

[10] A. Bompotas, N.-R. Kalogeropoulos, and C. Makris, ``CommC: A Multi-Purpose COMModity Hardware Cluster,'' *Future Internet*, vol. 17, no. 121, 2025.

[11] H. Gu, J. Wang, J. Yu, D. Wang, B. Li, X. He, and X. Yin, ``Towards virtual machine scheduling research based on multi-decision AHP method in the cloud computing platform,'' *PeerJ Computer Science*, vol. 9, e1675, 2023.

[12] S. Tuli, G. Casale, and N. R. Jennings, ``CILP: Co-simulation based Imitation Learner for Dynamic Resource Provisioning in Cloud Computing Environments,'' *arXiv preprint*, arXiv:2302.05630v2, 2023.

[13] Q.-M. Nguyen, L.-A. Phan, and T. Kim, ``Load-Balancing of Kubernetes-Based Edge Computing Infrastructure Using Resource Adaptive Proxy,'' *Sensors*, vol. 22, no. 2869, 2022.

[14] R. Yang, Z. Li, J. Qian, and Z. Li, ``Task-Driven Virtual Machine Optimization Placement Model and Algorithm,'' *Future Internet*, vol. 17, no. 73, 2025.

[15] Y. Alahmad and A. Agarwal, ``Multiple objectives dynamic VM placement for application service availability in cloud networks,'' *Journal of Cloud Computing*, vol. 13, no. 46, 2024.

[16] D. Saxena and A. K. Singh, ``A proactive autoscaling and energy-efficient VM allocation framework using online multi-resource neural network for cloud data center,'' *Neurocomputing*, 2022.